

“Vehicle Service Booking and Tracking System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

M SHASHANTH : 2520090129

M AKSHAY : 2520090065

G NANDHA KISHORE : 2520080050

Under the Esteemed Guidance of

Dr. M Gayatri

Associate Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled “Vehicle Service Booking and Tracking System” is a record of Bonafide work of M SHASHANTH - 2520090129, M AKSHAY - 2520090065, G NANDHA KISHORE - 2520080050 submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

M SHASHANTH – 2520090129

M AKSHAY – 2520090065

G NANDHA KISHORE – 2520080050

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is certify that the mini project based report entitled “Vehicle Service Booking and Tracking System” is a bonafide work done and submitted by M SHASHANTH - 2520090129, M AKSHAY - 2520090065, G NANDHA KISHORE - 2520080050 in partial fulfillment of the requirements for the award of the degree of BACHELOR OF TECHNOLOGY in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year 2024-2025.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator and my guide Dr. M Gayatri, Associate Professor, Department of Computer Science, which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)/Dr. Kaja Shareef (CSIT)**, Head of the *Department for Computer Science Engineering/ Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

M SHASHANTH – 2520090129

M AKSHAY – 2520090065

G NANDHA KISHORE – 2520080050

Abstract

The Vehicle Service Booking and Tracking System is a web-based platform that digitises the complete vehicle servicing process of a service center. Customers register vehicles, book services and track progress online, while service centers manage bookings, job cards, technician allocation and spare parts.

The system generates digital invoices, maintains complete service history, and supports notifications and online payments. It is built with React.js, Node.js, Express.js and MySQL, with Razorpay and Firebase Cloud Messaging as integrations. A normalized (3NF) relational schema with constraints and indexes stores all data, and a secure REST backend uses JWT authentication, role-based access control, password hashing, validation and rate limiting.

The goal of the project is to reduce manual work and improve service visibility and efficiency for both customers and the service center.

Index

S.No	Chapter	Topics	Page.no
1	Introduction	1.1 Background; 1.2 Problem statement; 1.3 Scope; 1.4 Objectives; 1.5 Importance; 1.6 Target users	7
2	System Requirements	2.1 Hardware; 2.2 Software; 2.3 Development tools	13
3	Technology Stack	3.1 Front-end; 3.2 Back-end; 3.3 Database; 3.4 Version control; 3.5 APIs / external services	16
4	System Architecture	4.1 Architecture diagram; 4.2 Layers; 4.3 Deployment architecture	19
5	Design	5.1 ER diagram; 5.2 Database schema; 5.3 Normalization; 5.4 Indexes and SQL	23
6	Implementation	6.1 Module-wise; 6.2 Front-end logic; 6.3 API integration; 6.4 Authentication	31
7	Features	7.1 Main features; 7.2 How each feature works	48
8	Testing	8.1 Postman; 8.2 Integration; 8.3 Tools; 8.4 Test cases and results	51
9	Deployment	9.1 Steps; 9.2 Environment configuration; 9.3 Hosting	56
10	Challenges & Limitations	10.1 Issues; 10.2 Solutions; 10.3 Limitations	59
11	Future Enhancements	11.1 Planned features; 11.2 Integrations / optimisations	61
12	Conclusion	12.1 Summary; 12.2 What was achieved; 12.3 Skills learned	62
13	References	Books, documentation sites used	64
14	Appendices	Screenshots, code snippets, setup, user manual, geo-tag photos, review forms	65

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

Vehicles are an everyday necessity for individuals, families and businesses, and regular servicing is essential for their safety, performance, fuel efficiency and resale value. A vehicle that is serviced on time breaks down less often, pollutes less and costs less to own over its lifetime. As the number of vehicles on the road keeps growing, service centers are expected to handle more customers, more vehicle models and a wider variety of services than ever before.

In spite of this growth, a large part of the daily work in many service centers is still done manually. Customers book a service by calling the center or by walking in, and the booking is written in a register or on a whiteboard. When the vehicle arrives, a paper job card is prepared and passed around the workshop. Technicians are assigned by the supervisor from memory, spare parts used are noted by hand, and the final bill is calculated manually at the counter. Each of these steps depends on people remembering, copying and re-entering the same information.

This way of working creates problems for both sides. A customer who has left a vehicle at the center has no simple way to know whether the work has started, whether an extra part is needed, or when the vehicle will be ready. Calling the center repeatedly for updates wastes the time of both the customer and the staff. For the service center, paper records are easily lost, difficult to search and impossible to analyse. When a customer returns after six months and asks what was done last time, someone has to search through files to answer.

The Vehicle Service Booking and Tracking System is a web-based platform that digitises the complete vehicle servicing process of a service center. Customers register their vehicles, book services and follow the progress of their vehicle online. The service center manages bookings, job cards, technician allocation and spare parts from the same platform. The system generates digital invoices, maintains a complete service history for every vehicle, and supports notifications and online payments.

The project is built with React.js on the front end, Node.js and Express.js on the back end, and MySQL as the relational database. Razorpay is integrated for online payments and Firebase Cloud Messaging (FCM) for notifications. It is developed as part of the course Database Systems Engineering and Distributed Backend Development (25CS1302E), and applies the concepts of that course, namely relational design, normalization, SQL, REST APIs and secure backend engineering, to a real-world problem.

From an academic and engineering perspective, the project combines the following core areas of computer science:

- 1 Database Management: ER modelling, a normalized (3NF) MySQL schema, constraints, indexes, joins, aggregates and transactions.

- 2 Web Technology: a React.js user interface for customers and service center staff.
- 3 Backend Engineering: a layered Node.js and Express.js REST API with JWT authentication and role-based access control.
- 4 API Integration: Razorpay for payments and Firebase Cloud Messaging for notifications.
- 5 Software Engineering Principles: modular and maintainable code, version control and systematic testing.

The remainder of this report describes how the problem was analysed, how the database and the backend were designed, how the system was implemented and tested, and how it can be extended in future.

1.2 Problem Statement

Vehicle service centers still depend on phone calls, walk-ins, paper job cards and manual billing. Customers cannot see the status of their vehicle, service history is hard to retrieve, and centers struggle to allocate technicians and track spare parts.

The proposed system provides one centralized platform where customers book and track services, and service centers manage job cards, technicians, parts, invoices and payments, all backed by a normalized relational database.

The specific problems identified during requirement analysis, and their effect on customers and the service center, are summarised in the following table.

Problem in the manual process	Effect
Bookings taken by phone call or walk-in and written in registers	Double bookings, forgotten appointments, long waiting time for customers
Paper job cards passed around the workshop	Cards are lost or damaged; no single view of the work in progress
No way for customers to see progress	Repeated phone calls; poor customer satisfaction
Technician allocation decided from memory	Uneven workload; some technicians idle while others are overloaded
Spare-part usage written by hand	Stock mismatches; parts used but not billed
Manual calculation of bills	Calculation errors, delays and disputes at the counter
Service history stored in paper files	Difficult to retrieve; no reliable record per vehicle

The problem is therefore not a lack of effort but a lack of a shared, structured record. A relational database that stores every customer, vehicle, booking, job card, invoice and payment in related tables, and a web application that gives each user the right view of that data, addresses the root of the problem.

1.3 Scope of the Project

The scope of the project covers the complete servicing workflow, from the moment a customer registers to the moment the invoice is paid and the record is added to the vehicle's service history.

In scope

- 1 A web application for vehicle service booking and tracking, with separate access for customers and the service center.
- 2 Customer registration, login and vehicle registration.
- 3 Service booking, job-card creation, technician allocation and service-status tracking (booked, in progress, completed).
- 4 Spare-part management linked to job cards.
- 5 Digital invoice generation, online payment through Razorpay and notifications through Firebase Cloud Messaging.
- 6 A complete service-history view for every vehicle.
- 7 A secure REST backend using JWT, role-based access control, password hashing, input validation, rate limiting and HTTPS.
- 8 A normalized MySQL database with constraints, indexes and transactions.

Out of scope (current version)

- Multi-branch or multi-company support.
- Microservices, caching and real-time vehicle tracking (listed as future work).
- Accounting, payroll and inventory purchasing for the service center.
- A native mobile application (the web interface is used on mobile browsers).

Assumptions

- The service center has a reliable internet connection.
- Customers have a valid email address and phone number.
- Services and their base prices are defined by the service center in advance.

1.4 Objective(s)

The objectives of the project are listed below, each with a short explanation of how it is addressed.

- 1 To develop a web application for vehicle service booking and tracking. A React.js front end and a Node.js and Express.js backend provide one platform for both customers and the service center.
- 2 To let customers register vehicles, book services and view service history. Customers can manage their own vehicles, create bookings and open a history page for each vehicle.
- 3 To manage job cards, technician allocation and service status updates. The service center creates one job card per booking, assigns a technician and updates the status as work proceeds.
- 4 To design a normalized MySQL schema (ER model, 3NF) with constraints and indexes. The schema removes redundancy and protects integrity with primary keys, foreign keys, UNIQUE and NOT NULL constraints.
- 5 To build a secure REST backend with JWT, role-based access control and validation. Every protected endpoint verifies a signed token and checks the role of the caller.
- 6 To generate digital invoices and support online payments and notifications. Invoices are generated from the service price and parts used, payments are made through Razorpay and notifications are sent through FCM.
- 7 To reduce manual work and improve service visibility for all users. Information entered once is available to everyone who needs it, instantly.

1.5 Importance of the Application

The application is important because it turns the scattered, paper-based records of a service center into one reliable, queryable source of truth. The benefits can be grouped by the people who gain from them.

For customers

- Book a service at any time without calling or visiting the center.
- See whether the vehicle is booked, in progress or completed, without asking.
- Receive notifications about important changes.
- View invoices, pay online and keep a permanent record of every service.

For the service center

- Less paperwork and fewer data-entry errors.
- A clear picture of daily bookings and technician workload.
- Spare parts linked to the job that used them, so billing is complete.

- Faster, more accurate billing and online collection of payments.
- Reports on bookings and revenue using simple SQL aggregates.

For academic purposes

The project demonstrates the practical use of the course topics. An ER model is converted into a normalized schema, DDL scripts, indexes and transactions; the data is exposed through secure REST APIs; and the whole system is tested with Postman. It also shows how a layered backend can later be split into microservices.

1.6 Target Users/Audience

User	Needs	How the system helps
Vehicle owner (customer)	Easy booking, visibility of progress, digital bills	Online booking, status tracking, invoices, online payment, service history
Service center staff / manager	Organise bookings, job cards, technicians, parts and billing	Dashboards of bookings, job-card management, technician allocation, invoicing
Technician	Know which jobs are assigned and update progress	Job cards with assigned vehicle and service, status updates
Small and medium service centers	An affordable system that can be customised	Open-source stack (React, Node.js, MySQL) that can be extended
Students and developers	A reference for database-driven web design	Documented schema, REST APIs and source code

1.7 Organisation of the Report

Chapter 2 lists the hardware and software requirements together with the functional and non-functional requirements. Chapter 3 describes the technology stack. Chapter 4 presents the system architecture, and Chapter 5 the database design. Chapter 6 explains the implementation module by module with code listings. Chapter 7 describes the features, Chapter 8 the testing, and Chapter 9 the deployment. Chapter 10 discusses challenges and limitations, Chapter 11 future enhancements and Chapter 12 the conclusion. References and appendices follow.

1.8 Development Approach and Feasibility

The system was developed iteratively. Requirements were analysed first, followed by the ER model and schema, then the core REST APIs, and then the service workflow, invoicing, payment and notification modules. Each increment was tested with Postman before the next was started.

Feasibility	Assessment
Technical	All technologies (React.js, Node.js, Express.js, MySQL) are mature and well documented; the team has the skills to use them, and Razorpay and FCM provide ready SDKs.
Economic	The core stack is open source and free to use. Razorpay charges a transaction fee only when payments are made. A modest computer is enough for development and a small deployment.

Feasibility	Assessment
Operational	The interface is simple enough for customers and staff to use with minimal training, and it replaces existing manual steps rather than adding new ones.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The system runs on an ordinary development or server machine. The minimum hardware configuration used during development and testing is given below.

Component	Minimum Requirement
Processor	Intel Core i3 or above
Memory (RAM)	Minimum 8 GB
Storage	20 GB free disk space
Network	Internet connection (required for Razorpay payments and FCM notifications)
Display / Input	Standard monitor, keyboard and mouse

The processor and memory figures are chosen because the development machine runs several programs at once: the code editor, the Node.js server, the React development server, MySQL Server, a browser and Postman. Eight gigabytes of memory allows all of them to run without swapping. The storage figure includes the operating system, the tools, the project's dependency folders and the database files.

2.2 Software Requirements

Software	Purpose
Windows 10/11	Operating system used for development
Node.js	JavaScript runtime for the backend
Express.js	Web framework for REST APIs
React.js	Front-end user interface library
MySQL Server	Relational database server
MySQL Workbench	Database design, ER modelling and query execution
Postman	REST API testing
Git and GitHub	Version control and collaboration
Web browser	Chrome, Edge or Firefox to run the front end

At run time the following Node.js packages are used by the backend: express (routing), mysql2 (database access), jsonwebtoken (JWT), bcrypt (password hashing), express-validator (input validation), express-rate-limit (rate limiting), helmet and cors (HTTP security headers and cross-origin control), dotenv (environment variables), razorpay (payment gateway SDK) and firebase-admin (Firebase Cloud Messaging).

2.3 Development Tools and Frameworks

- VS Code: the main code editor for the React front end and Node.js backend.
- MySQL Workbench: used to design the ER model, create tables (DDL), write SQL queries and inspect data.

- Postman: used to send requests to the REST endpoints and verify authentication, validation and status codes.
- Git and GitHub: used for version control, so every team member can work on the same code base safely.
- npm: package manager for installing Express.js, React.js and supporting libraries.

2.4 Functional Requirements

ID	Requirement
FR1	The system shall allow a new user to register with name, email, phone and password.
FR2	The system shall authenticate users with email and password and issue a JWT.
FR3	The system shall allow a customer to add, view and update their own vehicles.
FR4	The system shall list the services offered with their base prices.
FR5	The system shall allow a customer to book a service for a vehicle on a chosen date.
FR6	The system shall allow the service center to view all bookings, filtered by date and status.
FR7	The system shall allow the service center to create a job card for a booking.
FR8	The system shall allow the service center to allocate a technician to a job card.
FR9	The system shall allow the service center to update the status of a booking (booked, in progress, completed).
FR10	The system shall allow customers to view the current status of their bookings.
FR11	The system shall allow the service center to record the spare parts used on a job card.
FR12	The system shall reduce spare-part stock when a part is used.
FR13	The system shall generate a digital invoice for a completed job card.
FR14	The system shall allow the customer to pay an invoice online through Razorpay.
FR15	The system shall record every payment with its status.
FR16	The system shall send notifications through Firebase Cloud Messaging on booking and status changes.
FR17	The system shall show the complete service history of a vehicle.
FR18	The system shall restrict each API to the roles that are allowed to use it.

2.5 Non-Functional Requirements

Category	Requirement
Security	Passwords are stored only as bcrypt hashes; protected APIs require a valid JWT; communication uses HTTPS; inputs are validated; requests are rate limited.
Data integrity	Primary keys, foreign keys, UNIQUE and NOT NULL constraints prevent orphan and duplicate records; related updates run in transactions.
Performance	Indexes on booking_date and booking status keep listing queries fast; non-blocking I/O handles concurrent requests.
Usability	A responsive interface with simple forms; status shown clearly to the customer.
Maintainability	Layered router-controller-service structure; code kept in Git.

Category	Requirement
Scalability	Stateless backend (JWT) that can be containerised and later split into services.
Availability	Errors in external services (Razorpay, FCM) are handled so the core system keeps working.

2.6 Use Cases

Use case	Actor	Main flow
Register and log in	Customer, Service center	User submits details; backend stores hashed password; user logs in and receives JWT.
Register vehicle	Customer	Customer enters registration number and model; vehicle is saved against the customer.
Book service	Customer	Customer selects vehicle, service and date; booking is saved with status 'booked'.
Create job card	Service center	Staff opens a booking, creates a job card and assigns a technician.
Update status	Service center / Technician	Status moves to 'in progress' and then 'completed'; customer is notified.
Record spare parts	Service center	Parts and quantities are added to the job card; stock is reduced.
Generate invoice	Service center	System totals the service price and parts and stores the invoice.
Pay invoice	Customer	Customer pays through Razorpay; payment is verified and recorded.
View history	Customer, Service center	System lists all past services of a vehicle.

CHAPTER-3 TECHNOLOGY STACK

This chapter describes each technology used in the project and explains why it was chosen. The technologies work together as a layered stack: React.js in the browser, Express.js REST APIs on Node.js, and MySQL for storage, with Razorpay and Firebase Cloud Messaging as external services.

3.1 Front-End Design

The front end is developed with React.js. React uses a component-based approach, which lets the team build reusable pieces of the interface, such as the login form, vehicle list, booking form, status tracker and invoice page. Each component receives data and describes how it should look; when the data changes, React updates only the parts of the page that need to change. This makes the interface fast and keeps the code organised.

Two kinds of interface are provided. The customer interface is used to register vehicles, book services, view status and history, and pay invoices. The service center interface is used to manage bookings, job cards, technician allocation, spare parts and invoices. Both are served by the same React application, and the menus and pages shown depend on the role stored in the user's token.

Concepts used in the React code include functional components, the `useState` and `useEffect` hooks for local state and data loading, the `Context` API for sharing the logged-in user across the application, and client-side routing with protected routes that redirect unauthenticated users to the login page. All communication with the server is through asynchronous HTTP requests to the REST APIs.

3.2 Back-End Design

The backend is built with Node.js and Express.js. Node.js provides non-blocking, event-driven I/O using `async/await`, which suits an application that handles many short database and API requests. Express.js provides routing and middleware, so that common work such as authentication, validation and error handling is written once and applied to many routes.

The backend follows a layered architecture: Router, Middleware, Controller, Service Layer and finally the MySQL database. Routers map URLs to controllers; middleware handles authentication, role checks, validation and rate limiting; controllers read the request and send the response; and the service layer contains the business logic and the SQL queries. Errors are handled in one centralised error-handling middleware. Keeping the layers separate means that SQL is never mixed with request handling, which makes the code easier to test and to change.

Security is built into the backend with JWT-based authentication, `bcrypt` password hashing, role-based access control (RBAC), input validation, rate limiting and HTTPS.

3.3 Database Design

MySQL is used as the relational database. It stores customers, vehicles, services, bookings, job cards, technicians, spare parts, invoices, payments and device tokens. The schema is designed from an ER model and normalized up to the third normal form (3NF) to remove redundancy. Primary keys, foreign keys, UNIQUE and NOT NULL constraints protect data integrity, and indexes on booking_date and booking status speed up common queries.

The InnoDB storage engine is used because it supports transactions and foreign keys. Transactions are used so that operations that touch several tables, such as creating a job card and updating a booking, or creating an invoice, either complete fully or not at all. MySQL Workbench is used to draw the ER model, forward-engineer the DDL script and run test queries.

3.4 Version Control

Git is used to track every change to the code, and GitHub hosts the shared repository. Team members work on separate branches, commit regularly and merge changes after review. This keeps the history of the project, prevents accidental overwriting and makes it easy to return to an earlier working version. Secret values such as database passwords and API keys are kept in a .env file that is listed in .gitignore and is never committed.

3.5 APIs or External Services

Razorpay

Razorpay is the online payment gateway. The payment flow has three steps. First, the backend creates a Razorpay order for the invoice amount and stores a payment record with status 'created'. Second, the React front end opens the Razorpay checkout window with the order id, and the customer pays. Third, Razorpay returns a payment id and a signature; the backend verifies the signature using the secret key, marks the payment as successful and the invoice as paid. Because the secret key is used only on the server, it is never exposed to the browser.

Firebase Cloud Messaging (FCM)

Firebase Cloud Messaging delivers push notifications to the customer's device or browser. After login, the front end obtains an FCM device token and sends it to the backend, which stores it in the Device_Token table. When the booking is confirmed, the status changes or an invoice is ready, the backend sends a message to the stored token through the Firebase Admin SDK.

Internal REST APIs

The Auth, Vehicles, Bookings, Invoices and Payments APIs, together with supporting APIs for services, job cards and technicians, are consumed by the React front end. They are documented in Chapter 6.

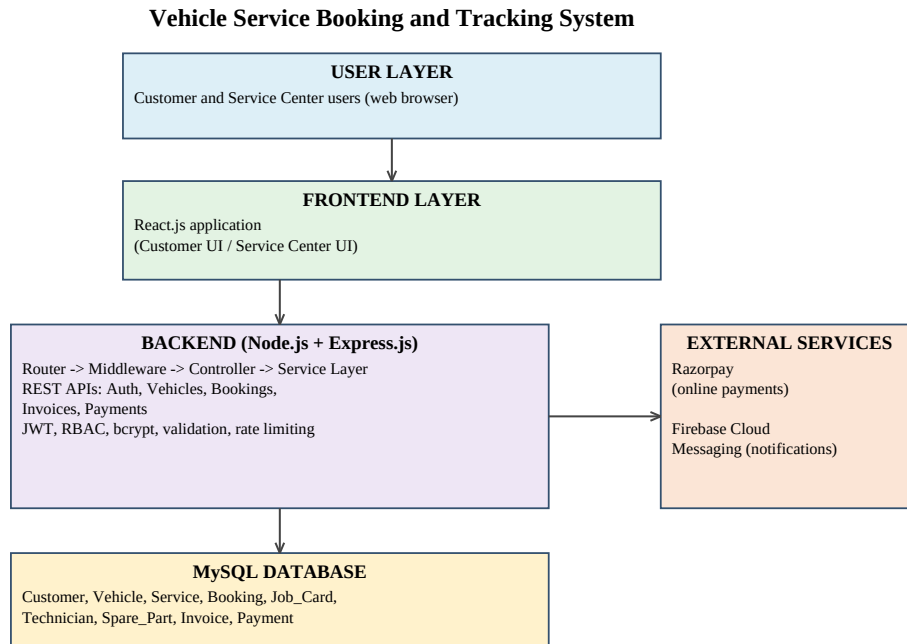
3.6 Summary of Technologies

Technology	Role in the Project	Why it was chosen
React.js	Frontend user interface	Reusable components, large community, fast updates

Technology	Role in the Project	Why it was chosen
Node.js + Express.js	Backend and REST APIs	Non-blocking I/O, simple routing and middleware
MySQL	Relational database (DDL, constraints, indexes)	Strong consistency, transactions, SQL support
JWT, bcrypt, RBAC	Authentication and security	Stateless tokens, safe password storage, role checks
Razorpay	Online payment gateway	Simple SDK and checkout for Indian payments
Firebase Cloud Messaging	Notifications	Free, reliable push delivery
Postman	API testing	Easy to create and save request collections
VS Code, MySQL Workbench	Development and database design	Productive tools for code and schema
Git and GitHub	Version control	Branching, history and collaboration

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-level architecture diagram



The system uses a layered, client-server architecture. The React.js frontend runs in the browser. It talks to the Node.js and Express.js backend through REST APIs. The backend reads and writes a MySQL database and calls two external services, Razorpay and Firebase Cloud Messaging.

4.2 Description of Each Layer (Frontend, Backend, Database)

Frontend layer

The React.js application is the only part the user sees. It shows the customer and service center interfaces, validates form input and sends requests to the backend REST APIs. It stores the JWT received at login and attaches it to every protected request. It never talks to the database directly and never holds secret keys.

Backend layer

The Node.js and Express.js server exposes REST APIs for Auth, Vehicles, Bookings, Invoices and Payments, together with supporting APIs for services, job cards and technicians. Each request passes through the Router, then the Middleware (authentication, role check, validation, rate limiting), then the Controller, and finally the Service Layer, which runs the business logic and talks to the database. Responses are returned as JSON with proper HTTP status codes.

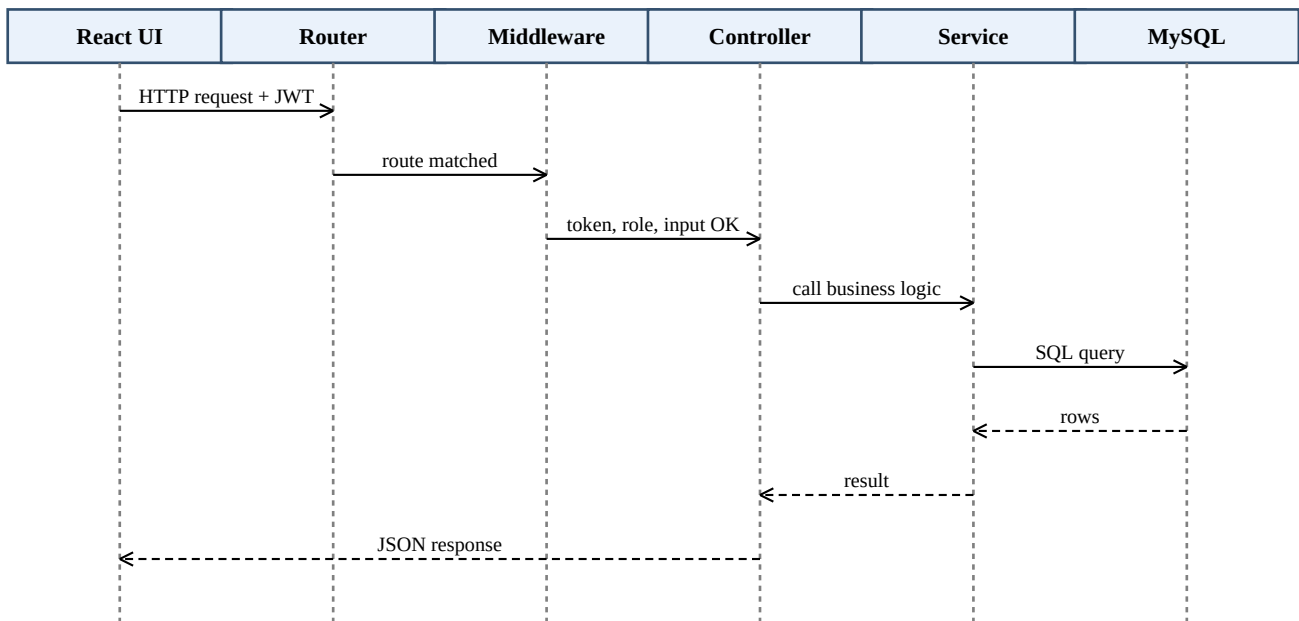


Figure: Life cycle of a request through the backend layers

Database layer

MySQL stores all persistent data in normalized tables. Foreign keys connect customers, vehicles, bookings, job cards, invoices and payments so that no orphan or duplicate records can exist. The database is accessed only from the service layer through a pooled connection.

External services

Razorpay handles online payments and Firebase Cloud Messaging delivers notifications. Both are called only from the backend, so secret keys never reach the browser.

4.3 Deployment Architecture (Cloud, Local, CI/CD Pipelines)

During development the complete system runs locally: the React development server, the Node.js and Express.js server, and MySQL Server on the same Windows machine, tested with Postman. The code is stored on GitHub.

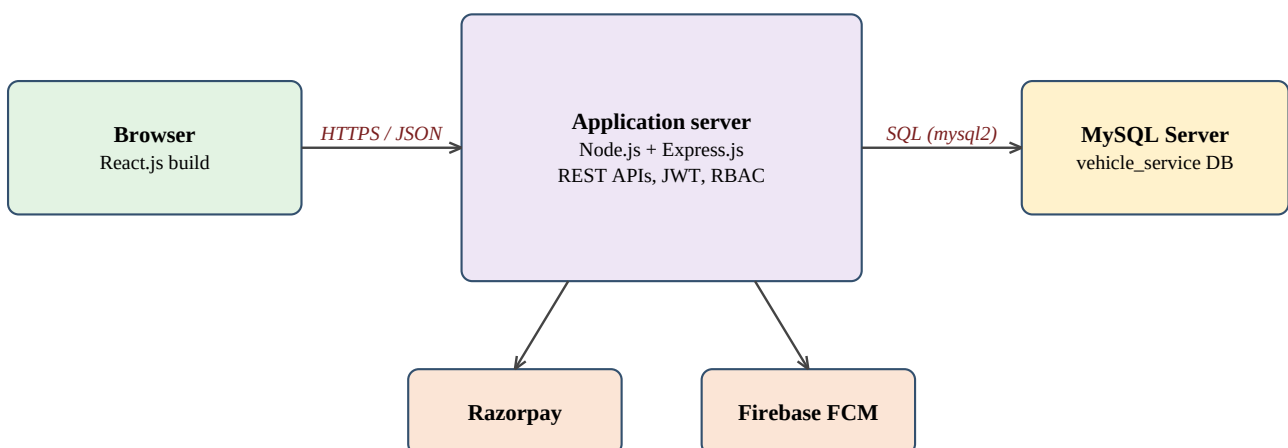


Figure: Deployment view

The planned deployment packages the backend and database in Docker containers so that the same configuration runs on any machine. In the future, Kubernetes, CI/CD pipelines and monitoring with Prometheus and Grafana can be added for larger deployments (see Chapter 11).

4.4 Workflow of the System

The overall workflow follows the proposed methodology. Users register and log in; the backend verifies credentials and issues a JWT. Customers add vehicles and book a service; the service center creates a job card and assigns a technician. Status updates, spare parts used and the final invoice are stored in MySQL. Payment is done via Razorpay and notifications are sent via FCM.

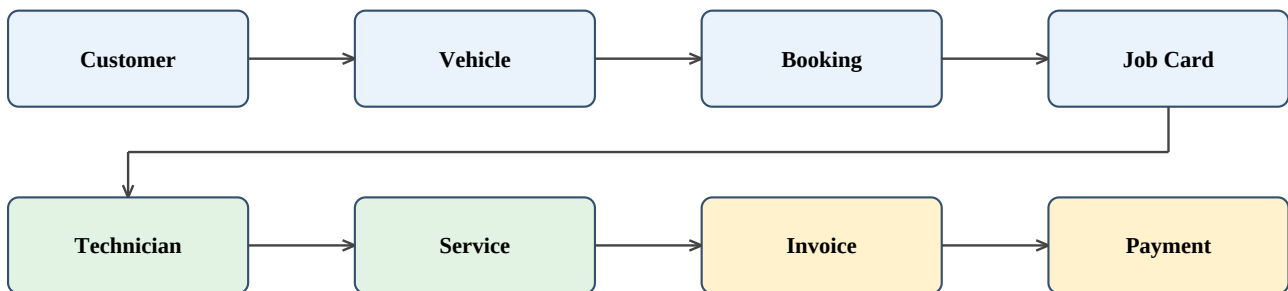


Figure: End-to-end workflow of the system

4.5 Data Flow (Level-0)

The context diagram shows the system as a single process with two main users and two external services. Customers send registration details, vehicle details and booking requests, and receive status information and invoices. The service center sends job-card, technician and status updates and receives bookings and reports. Razorpay receives payment orders, and FCM receives notification requests.

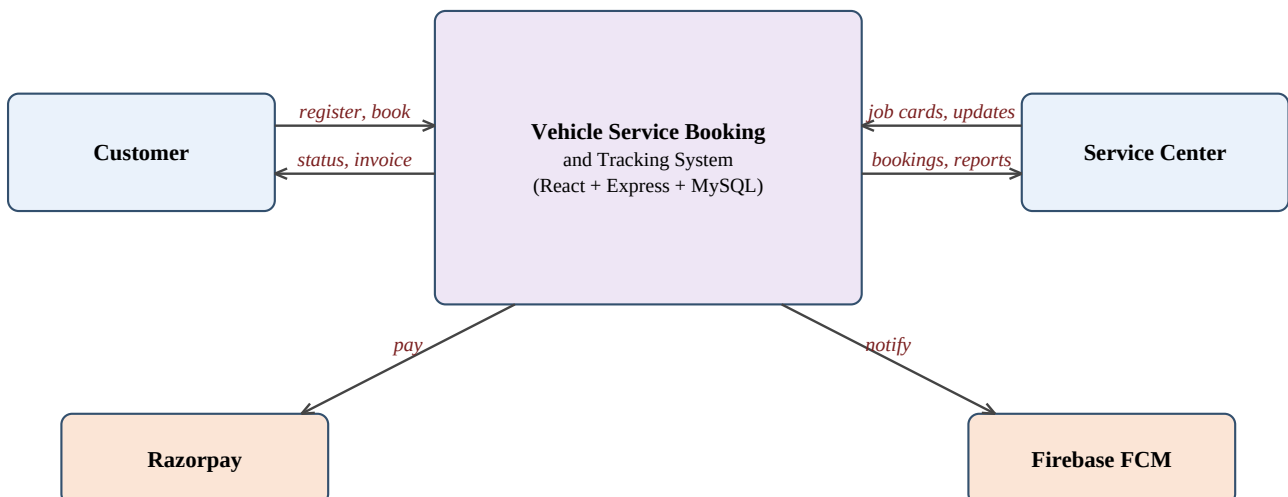


Figure: Context diagram (Level-0 data flow)

4.6 Design Principles Followed

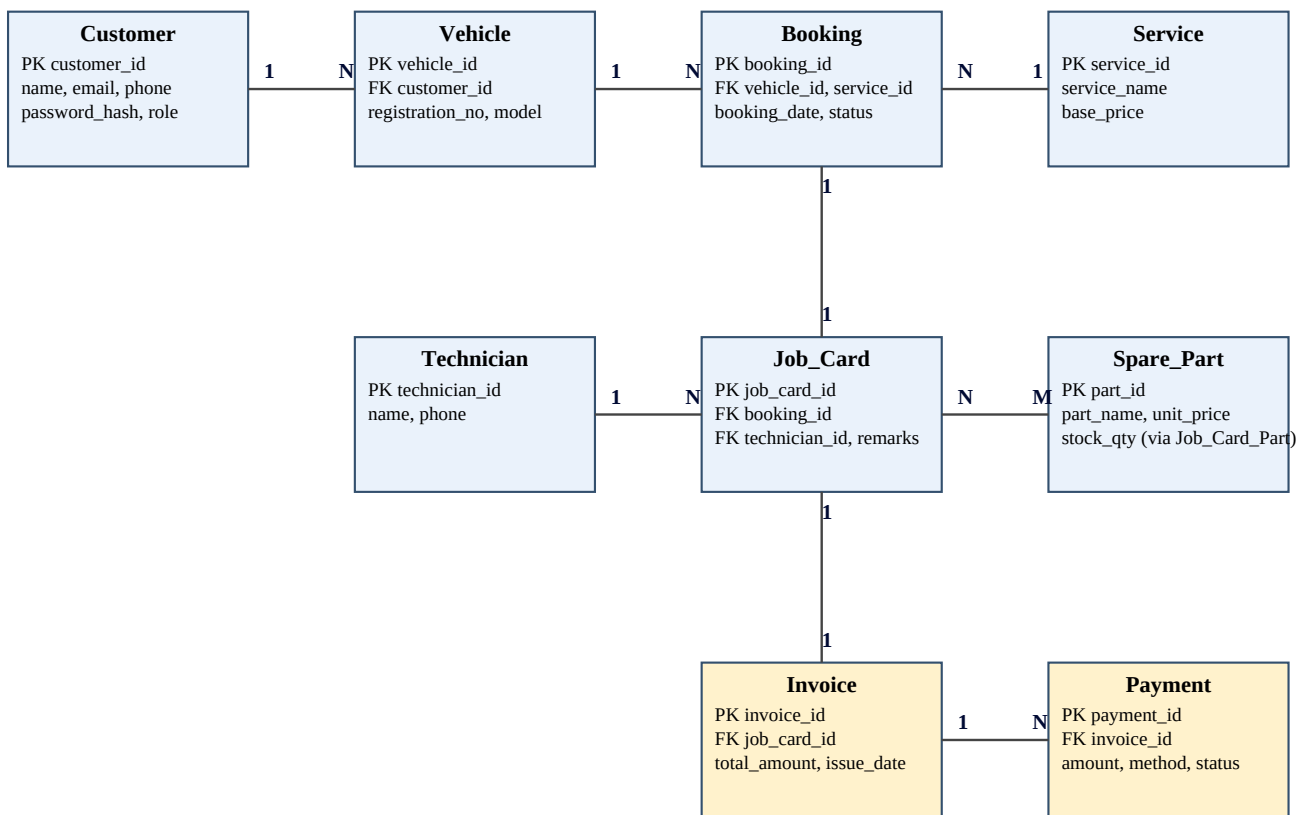
- Separation of concerns: routes, middleware, controllers and services each have one responsibility.

- Single source of truth: all data lives in MySQL; the front end holds only temporary state.
- Least privilege: each role can call only the APIs it needs.
- Fail safely: errors in external services are caught and reported without corrupting data.
- Configuration outside code: secrets and environment settings are read from environment variables.

CHAPTER -5 DESIGN

5.1 ER Diagram

The ER model contains the entities Customer, Vehicle, Service, Booking, Technician, Job_Card, Spare_Part, Invoice and Payment, with Job_Card_Part and Device_Token as supporting tables. The cardinalities are: Customer 1:N Vehicle, Vehicle 1:N Booking, Service 1:N Booking, Booking 1:1 Job_Card, Technician 1:N Job_Card, Job_Card 1:1 Invoice and Invoice 1:N Payment. Spare parts are linked to job cards through Job_Card_Part (many-to-many). In the diagram, 1 = one, N = many, PK = primary key and FK = foreign key.



1 = one, N = many, PK = primary key, FK = foreign key

5.2 Relationships

Relationship	Cardinality	Meaning
Customer - Vehicle	1 : N	One customer can register many vehicles; each vehicle belongs to one customer.
Vehicle - Booking	1 : N	A vehicle can be booked for service many times over its life.
Service - Booking	1 : N	A service (for example, oil change) appears in many bookings.
Booking - Job_Card	1 : 1	Each confirmed booking has exactly one job card.

Relationship	Cardinality	Meaning
Technician - Job_Card	1 : N	A technician can be allocated to many job cards over time.
Job_Card - Spare_Part	M : N	A job card can use many parts; a part is used in many job cards (Job_Card_Part).
Job_Card - Invoice	1 : 1	One invoice is generated for each completed job card.
Invoice - Payment	1 : N	An invoice can have more than one payment attempt.
Customer - Device_Token	1 : N	A customer can have several devices registered for notifications.

5.3 Database Schema (Data Dictionary)

The tables below describe every column of the schema. PK = primary key, FK = foreign key, NN = NOT NULL, UQ = UNIQUE, AI = AUTO_INCREMENT. Accounts of service center staff are stored in the Customer table with role = 'service_center'.

Table: Customer

Column	Data Type	Constraint	Description
customer_id	INT	PK, AI	Unique id of the user
name	VARCHAR(100)	NN	Full name
email	VARCHAR(120)	NN, UQ	Login email
phone	VARCHAR(15)	NN	Contact number
password_hash	VARCHAR(255)	NN	bcrypt hash of the password
role	ENUM	NN	'customer' or 'service_center'
created_at	TIMESTAMP	default now	Registration time

Table: Vehicle

Column	Data Type	Constraint	Description
vehicle_id	INT	PK, AI	Unique id of the vehicle
customer_id	INT	FK, NN	Owner (Customer)
registration_no	VARCHAR(20)	NN, UQ	Vehicle registration number
model	VARCHAR(50)	NN	Vehicle make and model
created_at	TIMESTAMP	default now	Registration time

Table: Service

Column	Data Type	Constraint	Description
service_id	INT	PK, AI	Unique id of the service
service_name	VARCHAR(80)	NN, UQ	Name of the service
base_price	DECIMAL(10,2)	NN	Base price of the service
description	VARCHAR(255)		Short description

Table: Booking

Column	Data Type	Constraint	Description
booking_id	INT	PK, AI	Unique id of the booking
vehicle_id	INT	FK, NN	Vehicle to be serviced
service_id	INT	FK, NN	Requested service
booking_date	DATE	NN, indexed	Date of the service
status	ENUM	NN, indexed	'booked', 'in_progress', 'completed'
created_at	TIMESTAMP	default now	Time the booking was made

Table: Technician

Column	Data Type	Constraint	Description
technician_id	INT	PK, AI	Unique id of the technician
name	VARCHAR(100)	NN	Technician name
phone	VARCHAR(15)		Contact number
is_available	TINYINT(1)	default 1	Whether the technician can take new work

Table: Job_Card

Column	Data Type	Constraint	Description
job_card_id	INT	PK, AI	Unique id of the job card
booking_id	INT	FK, NN, UQ	Booking this job card belongs to
technician_id	INT	FK	Allocated technician
remarks	TEXT		Notes about the work
opened_at	TIMESTAMP	default now	Time the job card was opened
closed_at	TIMESTAMP	NULL	Time the work was completed

Table: Spare_Part

Column	Data Type	Constraint	Description
part_id	INT	PK, AI	Unique id of the part
part_name	VARCHAR(100)	NN, UQ	Name of the part
unit_price	DECIMAL(10,2)	NN	Current price per unit
stock_qty	INT	NN, >= 0	Quantity in stock

Table: Job_Card_Part

Column	Data Type	Constraint	Description
job_card_id	INT	PK, FK	Job card that used the part
part_id	INT	PK, FK	Part used
quantity	INT	NN, > 0	Number of units used
unit_price	DECIMAL(10,2)	NN	Price per unit at the time of use

Table: Invoice

Column	Data Type	Constraint	Description
invoice_id	INT	PK, AI	Unique id of the invoice
job_card_id	INT	FK, NN, UQ	Job card billed
total_amount	DECIMAL(10,2)	NN	Service price plus parts
status	ENUM	NN	'unpaid' or 'paid'
issue_date	DATE	NN	Date the invoice was generated

Table: Payment

Column	Data Type	Constraint	Description
payment_id	INT	PK, AI	Unique id of the payment
invoice_id	INT	FK, NN	Invoice being paid
amount	DECIMAL(10,2)	NN	Amount paid
method	VARCHAR(30)		Payment method reported by Razorpay
razorpay_order_id	VARCHAR(60)	UQ	Razorpay order id
razorpay_payment_id	VARCHAR(60)		Razorpay payment id
payment_status	ENUM	NN	'created', 'success' or 'failed'
paid_at	TIMESTAMP	NULL	Time of successful payment

Table: Device_Token

Column	Data Type	Constraint	Description
token_id	INT	PK, AI	Unique id
customer_id	INT	FK, NN	Owner of the device
fcm_token	VARCHAR(255)	NN, UQ	Firebase device token

5.4 Normalization

The schema was normalized step by step, starting from a single table that mixed customer, vehicle, booking and billing details. Consider the following un-normalized row, in which one record repeats the same customer, vehicle and service information every time a part is used:

booking_id	customer	phone	vehicle_reg	service	price	parts_used
101	A. Rao	98xxxxxx01	TS09AB1234	Oil Change	800	Engine Oil, Oil Filter

First normal form (1NF)

The column parts_used holds more than one value, which violates 1NF. Each value is moved to its own row of a separate table (Job_Card_Part) so that every column holds a single, atomic value and every table has a primary key.

Second normal form (2NF)

2NF removes partial dependencies on part of a composite key. In Job_Card_Part, the key is (job_card_id, part_id). The quantity depends on both parts of the key and stays here, but the part name and current price depend only on part_id and are moved to Spare_Part.

Third normal form (3NF)

3NF removes dependencies between non-key columns. In the original table, customer name and phone depend on the customer, not on the booking; they are stored once in Customer. The service name and base price depend on the service, not on the booking; they are stored once in Service. Vehicle model depends on the vehicle and is stored in Vehicle. After this step every non-key column depends on the key, the whole key and nothing but the key. The unit_price stored in Job_Card_Part is intentional: it records the price actually charged at the time of use, so that later price changes do not alter old invoices.

5.5 DDL Scripts

The following script creates the database, all tables with their constraints, and the indexes. It is the script used in MySQL Workbench.

Listing 5.1: schema.sql

```
CREATE DATABASE IF NOT EXISTS vehicle_service CHARACTER SET utf8mb4;
USE vehicle_service;

CREATE TABLE Customer (
  customer_id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(120) NOT NULL UNIQUE,
  phone VARCHAR(15) NOT NULL,
  password_hash VARCHAR(255) NOT NULL,
  role ENUM('customer','service_center') NOT NULL DEFAULT 'customer',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
) ENGINE=InnoDB;

CREATE TABLE Vehicle (
  vehicle_id INT AUTO_INCREMENT PRIMARY KEY,
  customer_id INT NOT NULL,
  registration_no VARCHAR(20) NOT NULL UNIQUE,
  model VARCHAR(50) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
) ENGINE=InnoDB;

CREATE TABLE Service (
  service_id INT AUTO_INCREMENT PRIMARY KEY,
  service_name VARCHAR(80) NOT NULL UNIQUE,
  base_price DECIMAL(10,2) NOT NULL,
  description VARCHAR(255)
) ENGINE=InnoDB;

CREATE TABLE Booking (
  booking_id INT AUTO_INCREMENT PRIMARY KEY,
  vehicle_id INT NOT NULL,
  service_id INT NOT NULL,
  booking_date DATE NOT NULL,
  status ENUM('booked','in_progress','completed') NOT NULL DEFAULT 'booked',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (vehicle_id) REFERENCES Vehicle(vehicle_id),
  FOREIGN KEY (service_id) REFERENCES Service(service_id)
) ENGINE=InnoDB;

CREATE TABLE Technician (
```

```

    technician_id INT AUTO_INCREMENT PRIMARY KEY,
    name          VARCHAR(100) NOT NULL,
    phone         VARCHAR(15),
    is_available  TINYINT(1) NOT NULL DEFAULT 1
) ENGINE=InnoDB;

CREATE TABLE Job_Card (
    job_card_id  INT AUTO_INCREMENT PRIMARY KEY,
    booking_id   INT NOT NULL UNIQUE,
    technician_id INT,
    remarks      TEXT,
    opened_at    TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    closed_at    TIMESTAMP NULL,
    FOREIGN KEY (booking_id) REFERENCES Booking(booking_id),
    FOREIGN KEY (technician_id) REFERENCES Technician(technician_id)
) ENGINE=InnoDB;

CREATE TABLE Spare_Part (
    part_id      INT AUTO_INCREMENT PRIMARY KEY,
    part_name    VARCHAR(100) NOT NULL UNIQUE,
    unit_price   DECIMAL(10,2) NOT NULL,
    stock_qty    INT NOT NULL CHECK (stock_qty >= 0)
) ENGINE=InnoDB;

CREATE TABLE Job_Card_Part (
    job_card_id INT NOT NULL,
    part_id     INT NOT NULL,
    quantity    INT NOT NULL CHECK (quantity > 0),
    unit_price  DECIMAL(10,2) NOT NULL,
    PRIMARY KEY (job_card_id, part_id),
    FOREIGN KEY (job_card_id) REFERENCES Job_Card(job_card_id),
    FOREIGN KEY (part_id) REFERENCES Spare_Part(part_id)
) ENGINE=InnoDB;

CREATE TABLE Invoice (
    invoice_id  INT AUTO_INCREMENT PRIMARY KEY,
    job_card_id INT NOT NULL UNIQUE,
    total_amount DECIMAL(10,2) NOT NULL,
    status      ENUM('unpaid','paid') NOT NULL DEFAULT 'unpaid',
    issue_date  DATE NOT NULL,
    FOREIGN KEY (job_card_id) REFERENCES Job_Card(job_card_id)
) ENGINE=InnoDB;

CREATE TABLE Payment (
    payment_id      INT AUTO_INCREMENT PRIMARY KEY,
    invoice_id      INT NOT NULL,
    amount          DECIMAL(10,2) NOT NULL,
    method          VARCHAR(30),
    razorpay_order_id VARCHAR(60) UNIQUE,
    razorpay_payment_id VARCHAR(60),
    payment_status  ENUM('created','success','failed') NOT NULL DEFAULT 'created',
    paid_at         TIMESTAMP NULL,
    FOREIGN KEY (invoice_id) REFERENCES Invoice(invoice_id)
) ENGINE=InnoDB;

CREATE TABLE Device_Token (
    token_id      INT AUTO_INCREMENT PRIMARY KEY,
    customer_id   INT NOT NULL,
    fcm_token     VARCHAR(255) NOT NULL UNIQUE,
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id)
) ENGINE=InnoDB;

-- Indexes for frequent queries
CREATE INDEX idx_booking_date  ON Booking(booking_date);
CREATE INDEX idx_booking_status ON Booking(status);
CREATE INDEX idx_vehicle_owner ON Vehicle(customer_id);

```

5.6 Sample Data and Advanced SQL

The database was tested with sample customers, vehicles, services and bookings. The sample services and parts below are used for testing only.

Listing 5.2: sample data (seed.sql)

```
INSERT INTO Service (service_name, base_price, description) VALUES
('General Service', 1500.00, 'Full vehicle check and routine service'),
('Oil Change', 800.00, 'Engine oil and oil filter change'),
('Brake Inspection', 600.00, 'Brake pads, discs and fluid check'),
('Wheel Alignment', 700.00, 'Alignment and balancing'),
('Battery Check', 300.00, 'Battery health and terminals');

INSERT INTO Technician (name, phone) VALUES
('Technician One', '9000000001'),
('Technician Two', '9000000002'),
('Technician Three', '9000000003');

INSERT INTO Spare_Part (part_name, unit_price, stock_qty) VALUES
('Engine Oil 1L', 450.00, 50),
('Oil Filter', 250.00, 40),
('Brake Pad Set', 1800.00, 15),
('Air Filter', 350.00, 25);
```

The queries below show the joins, aggregates and transactions used by the application.

Listing 5.3: queries.sql

```
-- 1. Service history of one vehicle (join)
SELECT b.booking_date, s.service_name, b.status, i.total_amount
FROM Booking b
JOIN Service s      ON s.service_id = b.service_id
LEFT JOIN Job_Card j ON j.booking_id = b.booking_id
LEFT JOIN Invoice i  ON i.job_card_id = j.job_card_id
WHERE b.vehicle_id = ?
ORDER BY b.booking_date DESC;

-- 2. Bookings of a day by status (uses idx_booking_date, idx_booking_status)
SELECT b.booking_id, v.registration_no, s.service_name, b.status
FROM Booking b
JOIN Vehicle v ON v.vehicle_id = b.vehicle_id
JOIN Service s ON s.service_id = b.service_id
WHERE b.booking_date = ? AND b.status = ?;

-- 3. Number of bookings per service (aggregate)
SELECT s.service_name, COUNT(*) AS total_bookings
FROM Booking b JOIN Service s ON s.service_id = b.service_id
GROUP BY s.service_name
ORDER BY total_bookings DESC;

-- 4. Revenue per day (aggregate)
SELECT DATE(p.paid_at) AS day, SUM(p.amount) AS revenue
FROM Payment p
WHERE p.payment_status = 'success'
GROUP BY DATE(p.paid_at)
ORDER BY day;

-- 5. Workload of each technician
SELECT t.name, COUNT(j.job_card_id) AS open_jobs
FROM Technician t
LEFT JOIN Job_Card j ON j.technician_id = t.technician_id AND j.closed_at IS NULL
GROUP BY t.technician_id, t.name;

-- 6. Parts that need re-ordering
SELECT part_name, stock_qty FROM Spare_Part WHERE stock_qty < 10;
```

```
-- 7. Transaction: create a job card and start the booking
START TRANSACTION;
  INSERT INTO Job_Card (booking_id, technician_id) VALUES (?, ?);
  UPDATE Booking SET status = 'in_progress' WHERE booking_id = ?;
COMMIT;
```

5.7 Constraints, Indexes and Transactions

- Primary keys uniquely identify every row; AUTO_INCREMENT keys are generated by the database.
- Foreign keys make sure a vehicle always has an owner, a booking always refers to an existing vehicle and service, and so on; rows that are still referenced cannot be deleted.
- UNIQUE constraints on email, registration_no, service_name and part_name prevent duplicates; UNIQUE on Job_Card.booking_id and Invoice.job_card_id enforce the one-to-one relationships.
- NOT NULL and CHECK constraints reject incomplete or invalid data, such as negative stock.
- The indexes on booking_date and status make the frequent 'bookings of the day by status' query faster, and the index on Vehicle.customer_id speeds up 'my vehicles'.
- Transactions are used where several tables change together: creating a job card with the booking status, recording parts with the stock reduction, and generating an invoice. If any step fails the whole transaction is rolled back.



Figure: Lifecycle of a service request

The status of a service request follows the lifecycle shown above: a booking is created as 'booked', moves to 'in_progress' when the job card is opened, and to 'completed' when the work is finished. After completion an invoice is generated and finally paid.

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

The system is implemented in three modules. Module 1 is the database, Module 2 is the backend REST API with security, and Module 3 is the service workflow with invoicing and payments. The React front end consumes the APIs of Modules 2 and 3.

Module 1: Database Design (ER Model, Schema and SQL)

The ER model was drawn first with entities Customer, Vehicle, Service and Booking, and then extended with job cards, technicians, spare parts, invoices and payments. The relational schema was written with primary keys, foreign keys, UNIQUE and NOT NULL constraints and normalized from 1NF to 3NF. The DDL script (Listing 5.1) contains statements such as CREATE TABLE Vehicle and the indexes on booking_date and booking status. Advanced SQL covers joins for service history, aggregates for reports and transactions for booking and billing (Listing 5.3). The complete design is described in Chapter 5.

Module 2: Backend REST API and Security (Node.js + Express)

The backend uses the layered structure Router, Middleware, Controller, Service Layer and MySQL. The folder structure of the backend project is shown below.

```
backend/
  package.json
  .env                (not committed to GitHub)
  src/
    app.js            Express app, global middleware, route mounting
    server.js         Starts the HTTP server
    config/db.js      MySQL connection pool
    middleware/
      auth.js         verifyToken, authorize(roles)
      validate.js     express-validator rules and result handler
      rateLimiter.js  Rate limiting
      errorHandler.js Centralised error handling
    routes/           auth, vehicles, bookings, jobcards, invoices, payments
    controllers/      Read request, call service, send response
    services/         Business logic and SQL
      fcm.service.js  Firebase Cloud Messaging
      razorpay.service.js Razorpay orders and signature check
```

The main endpoints are POST /api/auth/register, POST /api/auth/login, GET and POST /api/vehicles, POST /api/bookings and PUT /api/bookings/:id/status, followed by POST /api/invoices and POST /api/payments. All of them are documented in section 6.3. The following listings show the main parts of the backend code.

Listing 6.1: src/config/db.js

```
const mysql = require('mysql2/promise');
require('dotenv').config();

const pool = mysql.createPool({
  host: process.env.DB_HOST,
  user: process.env.DB_USER,
  password: process.env.DB_PASSWORD,
  database: process.env.DB_NAME,
  waitForConnections: true,
```

```

    connectionLimit: 10,
  });

```

```

module.exports = pool;

```

Listing 6.2: src/app.js

```

const express = require('express');
const cors = require('cors');
const helmet = require('helmet');
require('dotenv').config();

const { apiLimiter } = require('./middleware/rateLimiter');
const errorHandler = require('./middleware/errorHandler');

const app = express();
app.use(helmet()); // secure HTTP headers
app.use(cors({ origin: process.env.CLIENT_URL }));
app.use(express.json());
app.use('/api', apiLimiter); // rate limiting

app.use('/api/auth', require('./routes/auth.routes'));
app.use('/api/vehicles', require('./routes/vehicle.routes'));
app.use('/api/services', require('./routes/service.routes'));
app.use('/api/bookings', require('./routes/booking.routes'));
app.use('/api/technicians', require('./routes/technician.routes'));
app.use('/api/jobcards', require('./routes/jobcard.routes'));
app.use('/api/invoices', require('./routes/invoice.routes'));
app.use('/api/payments', require('./routes/payment.routes'));

app.use((req, res) => res.status(404).json({ message: 'Route not found' }));
app.use(errorHandler); // must be last

module.exports = app;

```

Listing 6.3: src/server.js

```

const app = require('./app');

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on port ${PORT}`));

```

Listing 6.4: src/middleware/rateLimiter.js and errorHandler.js

```

// rateLimiter.js
const rateLimit = require('express-rate-limit');

exports.apiLimiter = rateLimit({
  windowMs: 15 * 60 * 1000, // 15 minutes
  max: 300, // requests per IP per window
  standardHeaders: true,
  legacyHeaders: false,
});

exports.loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 10, // stricter limit for login attempts
  message: { message: 'Too many login attempts, try again later' },
});

// errorHandler.js
module.exports = (err, req, res, next) => {
  console.error(err);
  const status = err.status || 500;
  res.status(status).json({
    message: status === 500 ? 'Internal server error' : err.message,
  });
};

```

Listing 6.5: src/middleware/validate.js

```
const { body, param, validationResult } = require('express-validator');

exports.handle = (req, res, next) => {
  const errors = validationResult(req);
  if (!errors.isEmpty()) {
    return res.status(400).json({ message: 'Validation failed', errors: errors.array() });
  }
  next();
};

exports.registerRules = [
  body('name').trim().notEmpty(),
  body('email').isEmail().normalizeEmail(),
  body('phone').isMobilePhone(),
  body('password').isLength({ min: 8 }),
];

exports.loginRules = [body('email').isEmail(), body('password').notEmpty()];

exports.vehicleRules = [
  body('registration_no').trim().notEmpty().isLength({ max: 20 }),
  body('model').trim().notEmpty(),
];

exports.bookingRules = [
  body('vehicle_id').isInt({ min: 1 }),
  body('service_id').isInt({ min: 1 }),
  body('booking_date').isISO8601().toDate(),
];

exports.statusRules = [
  param('id').isInt({ min: 1 }),
  body('status').isIn(['booked', 'in_progress', 'completed']),
];
```

Listing 6.6: src/routes/auth.routes.js and vehicle.routes.js

```
// auth.routes.js
const router = require('express').Router();
const ctrl = require('../controllers/auth.controller');
const v = require('../middleware/validate');
const { verifyToken } = require('../middleware/auth');
const { loginLimiter } = require('../middleware/rateLimiter');

router.post('/register', v.registerRules, v.handle, ctrl.register);
router.post('/login', loginLimiter, v.loginRules, v.handle, ctrl.login);
router.post('/device-token', verifyToken, ctrl.saveDeviceToken);
module.exports = router;

// vehicle.routes.js
const router2 = require('express').Router();
const v = require('../middleware/validate');
const vehicles = require('../controllers/vehicle.controller');
const { verifyToken, authorize } = require('../middleware/auth');

router2.use(verifyToken);
router2.get('/', authorize('customer'), vehicles.list);
router2.post('/', authorize('customer'), v.vehicleRules, v.handle, vehicles.create);
router2.get('/:id/history', vehicles.history);
module.exports = router2;
```

Listing 6.7: src/routes/booking.routes.js

```
const router = require('express').Router();
const bookings = require('../controllers/booking.controller');
```

```

const v = require('../middleware/validate');
const { verifyToken, authorize } = require('../middleware/auth');

router.use(verifyToken);
router.post('/', authorize('customer'), v.bookingRules, v.handle, bookings.create);
router.get('/', bookings.list);
router.put('/:id/status', authorize('service_center'), v.statusRules, v.handle,
    bookings.updateStatus);
module.exports = router;

```

Listing 6.8: src/controllers/auth.controller.js

```

const authService = require('../services/auth.service');

exports.register = async (req, res, next) => {
  try {
    const id = await authService.register(req.body);
    res.status(201).json({ message: 'Registered', customer_id: id });
  } catch (err) { next(err); }
};

exports.login = async (req, res, next) => {
  try {
    const result = await authService.login(req.body);
    res.json(result);
  } catch (err) { next(err); }
};

exports.saveDeviceToken = async (req, res, next) => {
  try {
    await authService.saveDeviceToken(req.user.id, req.body.fcm_token);
    res.status(201).json({ message: 'Token saved' });
  } catch (err) { next(err); }
};

```

Listing 6.9: src/services/auth.service.js

```

const bcrypt = require('bcrypt');
const jwt = require('jsonwebtoken');
const db = require('../config/db');

const httpError = (status, message) => Object.assign(new Error(message), { status });

exports.register = async ({ name, email, phone, password }) => {
  const [existing] = await db.query('SELECT 1 FROM Customer WHERE email = ?', [email]);
  if (existing.length) throw httpError(409, 'Email already registered');

  const hash = await bcrypt.hash(password, 10);
  const [result] = await db.query(
    'INSERT INTO Customer (name, email, phone, password_hash) VALUES (?, ?, ?, ?)',
    [name, email, phone, hash]
  );
  return result.insertId;
};

exports.login = async ({ email, password }) => {
  const [rows] = await db.query('SELECT * FROM Customer WHERE email = ?', [email]);
  const user = rows[0];
  if (!user || !(await bcrypt.compare(password, user.password_hash))) {
    throw httpError(401, 'Invalid email or password');
  }
  const token = jwt.sign(
    { id: user.customer_id, role: user.role },
    process.env.JWT_SECRET,
    { expiresIn: '8h' }
  );
  return { token, user: { customer_id: user.customer_id, name: user.name, role: user.role } };
};

```

```

exports.saveDeviceToken = async (customerId, token) => {
  await db.query(
    'INSERT IGNORE INTO Device_Token (customer_id, fcm_token) VALUES (?, ?)',
    [customerId, token]
  );
};

```

Listing 6.10: src/middleware/auth.js (authentication and RBAC)

```

const jwt = require('jsonwebtoken');

exports.verifyToken = (req, res, next) => {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return res.status(401).json({ message: 'Authentication required' });
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET); // { id, role }
    next();
  } catch (err) {
    return res.status(401).json({ message: 'Invalid or expired token' });
  }
};

exports.authorize = (...roles) => (req, res, next) => {
  if (!roles.includes(req.user.role)) {
    return res.status(403).json({ message: 'Access denied' });
  }
  next();
};

```

Listing 6.11: src/services/vehicle.service.js

```

const db = require('../config/db');
const httpError = (s, m) => Object.assign(new Error(m), { status: s });

exports.list = async (customerId) => {
  const [rows] = await db.query(
    'SELECT vehicle_id, registration_no, model FROM Vehicle WHERE customer_id = ?',
    [customerId]
  );
  return rows;
};

exports.create = async (customerId, { registration_no, model }) => {
  try {
    const [r] = await db.query(
      'INSERT INTO Vehicle (customer_id, registration_no, model) VALUES (?, ?, ?)',
      [customerId, registration_no, model]
    );
    return r.insertId;
  } catch (err) {
    if (err.code === 'ER_DUP_ENTRY') throw httpError(409, 'Registration number exists');
    throw err;
  }
};

exports.history = async (user, vehicleId) => {
  const [v] = await db.query('SELECT customer_id FROM Vehicle WHERE vehicle_id = ?', [vehicleId]);
  if (!v.length) throw httpError(404, 'Vehicle not found');
  if (user.role === 'customer' && v[0].customer_id !== user.id) throw httpError(403, 'Not allowed');

  const [rows] = await db.query(
    `SELECT b.booking_date, s.service_name, b.status, i.total_amount
    FROM Booking b
    JOIN Service s      ON s.service_id = b.service_id
    LEFT JOIN Job_Card j ON j.booking_id = b.booking_id
    LEFT JOIN Invoice i  ON i.job_card_id = j.job_card_id`
  );
};

```

```

        WHERE b.vehicle_id = ?
        ORDER BY b.booking_date DESC`,
    [vehicleId]
  );
  return rows;
};

```

Listing 6.12: src/services/booking.service.js

```

const db = require('../config/db');
const fcm = require('../fcm.service');
const httpError = (s, m) => Object.assign(new Error(m), { status: s });

exports.create = async (user, { vehicle_id, service_id, booking_date }) => {
  const [v] = await db.query(
    'SELECT 1 FROM Vehicle WHERE vehicle_id = ? AND customer_id = ?',
    [vehicle_id, user.id]
  );
  if (!v.length) throw httpError(403, 'Vehicle does not belong to you');
  const [s] = await db.query('SELECT 1 FROM Service WHERE service_id = ?', [service_id]);
  if (!s.length) throw httpError(404, 'Service not found');

  const [r] = await db.query(
    'INSERT INTO Booking (vehicle_id, service_id, booking_date) VALUES (?, ?, ?)',
    [vehicle_id, service_id, booking_date]
  );
  return r.insertId;
};

exports.list = async (user, { date, status }) => {
  let sql = `SELECT b.booking_id, v.registration_no, s.service_name,
    b.booking_date, b.status
    FROM Booking b
    JOIN Vehicle v ON v.vehicle_id = b.vehicle_id
    JOIN Service s ON s.service_id = b.service_id WHERE 1=1`;
  const params = [];
  if (user.role === 'customer') { sql += ' AND v.customer_id = ?'; params.push(user.id); }
  if (date) { sql += ' AND b.booking_date = ?'; params.push(date); }
  if (status) { sql += ' AND b.status = ?'; params.push(status); }
  sql += ' ORDER BY b.booking_date DESC';
  const [rows] = await db.query(sql, params);
  return rows;
};

exports.updateStatus = async (bookingId, status) => {
  const [r] = await db.query('UPDATE Booking SET status = ? WHERE booking_id = ?',
    [status, bookingId]);
  if (!r.affectedRows) throw httpError(404, 'Booking not found');

  const [[row]] = await db.query(
    `SELECT v.customer_id FROM Booking b
    JOIN Vehicle v ON v.vehicle_id = b.vehicle_id WHERE b.booking_id = ?`,
    [bookingId]
  );
  await fcm.notifyCustomer(row.customer_id, 'Service update',
    `Your booking #${bookingId} is now ${status}.`);
};

```

Module 3: Service Workflow, Invoicing and Payments

For each confirmed booking the service center creates a job card and allocates a technician. The service status (booked, in progress, completed) is updated as work proceeds and is visible to the customer. Spare parts used are linked to the job card. When the work is complete, a digital invoice is generated through POST /api/invoices, the customer pays online through Razorpay (POST /api/payments), notifications are

sent through Firebase Cloud Messaging, and the full record becomes part of the vehicle's service history.

Listing 6.13: src/services/jobcard.service.js (transactions)

```
const db = require('../config/db');
const httpError = (s, m) => Object.assign(new Error(m), { status: s });

exports.create = async ({ booking_id, technician_id }) => {
  const conn = await db.getConnection();
  try {
    await conn.beginTransaction();
    const [b] = await conn.query('SELECT status FROM Booking WHERE booking_id = ? FOR UPDATE',
                                  [booking_id]);
    if (!b.length) throw httpError(404, 'Booking not found');

    const [r] = await conn.query(
      'INSERT INTO Job_Card (booking_id, technician_id) VALUES (?, ?)',
      [booking_id, technician_id]
    );
    await conn.query("UPDATE Booking SET status = 'in_progress' WHERE booking_id = ?",
                     [booking_id]);
    await conn.commit();
    return r.insertId;
  } catch (err) {
    await conn.rollback();
    if (err.code === 'ER_DUP_ENTRY') throw httpError(409, 'Job card already exists');
    throw err;
  } finally {
    conn.release();
  }
};

exports.addPart = async (jobCardId, { part_id, quantity }) => {
  const conn = await db.getConnection();
  try {
    await conn.beginTransaction();
    const [p] = await conn.query(
      'SELECT unit_price, stock_qty FROM Spare_Part WHERE part_id = ? FOR UPDATE', [part_id]);
    if (!p.length) throw httpError(404, 'Part not found');
    if (p[0].stock_qty < quantity) throw httpError(400, 'Insufficient stock');

    await conn.query(
      'INSERT INTO Job_Card_Part (job_card_id, part_id, quantity, unit_price) VALUES (?, ?, ?, ?)',
      [jobCardId, part_id, quantity, p[0].unit_price]
    );
    await conn.query('UPDATE Spare_Part SET stock_qty = stock_qty - ? WHERE part_id = ?',
                     [quantity, part_id]);
    await conn.commit();
  } catch (err) {
    await conn.rollback();
    throw err;
  } finally {
    conn.release();
  }
};
```

Listing 6.14: src/services/invoice.service.js

```
const db = require('../config/db');
const fcm = require('../fcm.service');
const httpError = (s, m) => Object.assign(new Error(m), { status: s });

exports.create = async ({ job_card_id }) => {
  const conn = await db.getConnection();
  try {
    await conn.beginTransaction();
    const [[job]] = await conn.query(
```

```

    `SELECT s.base_price, v.customer_id
      FROM Job_Card j
      JOIN Booking b ON b.booking_id = j.booking_id
      JOIN Service s ON s.service_id = b.service_id
      JOIN Vehicle v ON v.vehicle_id = b.vehicle_id
     WHERE j.job_card_id = ?`, [job_card_id]);
    if (!job) throw httpError(404, 'Job card not found');

    const [[parts]] = await conn.query(
      `SELECT COALESCE(SUM(quantity * unit_price), 0) AS parts_total
      FROM Job_Card_Part WHERE job_card_id = ?`, [job_card_id]);

    const total = Number(job.base_price) + Number(parts.parts_total);
    const [r] = await conn.query(
      `INSERT INTO Invoice (job_card_id, total_amount, issue_date) VALUES (?, ?, CURDATE())`,
      [job_card_id, total]
    );
    await conn.query(
      `UPDATE Booking SET status = 'completed'
      WHERE booking_id = (SELECT booking_id FROM Job_Card WHERE job_card_id = ?)`,
      [job_card_id]
    );
    await conn.query(`UPDATE Job_Card SET closed_at = NOW() WHERE job_card_id = ?`, [
job_card_id]);
    await conn.commit();

    await fcm.notifyCustomer(job.customer_id, 'Invoice ready',
      `Your invoice of Rs. ${total} is ready.`);
    return { invoice_id: r.insertId, total_amount: total, status: 'unpaid' };
  } catch (err) {
    await conn.rollback();
    if (err.code === 'ER_DUP_ENTRY') throw httpError(409, 'Invoice already exists');
    throw err;
  } finally {
    conn.release();
  }
};

```

6.2 Front-end Logic (UI Rendering, State Management)

The React interface is divided into components such as Login, Register, VehicleList, BookingForm, StatusTracker, InvoiceView and PaymentButton. Component state, kept with the `useState` hook, holds form values and the data returned by the API; the `useEffect` hook loads data when a page opens. After login the JWT is kept in the `AuthContext` and sent in the Authorization header of every request. The interface shows different menus depending on whether the logged-in user is a customer or service center staff, and routes that need a login are wrapped in a `ProtectedRoute` component. The front-end integration with the backend is still being completed (see Chapter 10).

Listing 6.15: src/api.js and src/AuthContext.jsx

```

// api.js
import axios from 'axios';

const api = axios.create({ baseURL: process.env.REACT_APP_API_URL });

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('token');
  if (token) config.headers.Authorization = `Bearer ${token}`;
  return config;
});

export default api;

```

```
// AuthContext.jsx
import { createContext, useContext, useState } from 'react';
import api from './api';

const AuthContext = createContext(null);
export const useAuth = () => useContext(AuthContext);

export function AuthProvider({ children }) {
  const [user, setUser] = useState(JSON.parse(localStorage.getItem('user')));

  const login = async (email, password) => {
    const { data } = await api.post('/api/auth/login', { email, password });
    localStorage.setItem('token', data.token);
    localStorage.setItem('user', JSON.stringify(data.user));
    setUser(data.user);
  };

  const logout = () => { localStorage.clear(); setUser(null); };

  return <AuthContext.Provider value={{ user, login, logout }}>{children}</AuthContext.Provider>;
}
```

Listing 6.16: src/App.jsx (routing and protected routes)

```
import { BrowserRouter, Routes, Route, Navigate } from 'react-router-dom';
import { AuthProvider, useAuth } from './AuthContext';
import Login from './pages/Login';
import VehicleList from './pages/VehicleList';
import BookingForm from './pages/BookingForm';
import StatusTracker from './pages/StatusTracker';
import InvoiceView from './pages/InvoiceView';

function Protected({ roles, children }) {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" />;
  if (roles && !roles.includes(user.role)) return <Navigate to="/" />;
  return children;
}

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Routes>
          <Route path="/login" element={<Login />} />
          <Route path="/vehicles" element={<Protected roles={['customer']><VehicleList /></Protected>} />
          <Route path="/book" element={<Protected roles={['customer']><BookingForm /></Protected>} />
          <Route path="/status" element={<Protected><StatusTracker /></Protected>} />
          <Route path="/invoice/:id" element={<Protected><InvoiceView /></Protected>} />
          <Route path="*" element={<Navigate to="/status" />} />
        </Routes>
      </BrowserRouter>
    </AuthProvider>
  );
}
```

Listing 6.17: src/pages/Login.jsx

```
import { useState } from 'react';
import { useNavigate } from 'react-router-dom';
import { useAuth } from '../AuthContext';

export default function Login() {
  const { login } = useAuth();
  const navigate = useNavigate();
  const [email, setEmail] = useState('');
```

```

const [password, setPassword] = useState('');
const [error, setError] = useState('');

const submit = async () => {
  try {
    await login(email, password);
    navigate('/status');
  } catch (err) {
    setError(err.response?.data?.message || 'Login failed');
  }
};

return (
  <div className="card">
    <h2>Vehicle Service Login</h2>
    <input placeholder="Email" value={email} onChange={(e) => setEmail(e.target.value)} />
    <input type="password" placeholder="Password" value={password}
      onChange={(e) => setPassword(e.target.value)} />
    {error && <p className="error">{error}</p>}
    <button onClick={submit}>Login</button>
  </div>
);
}

```

Listing 6.18: src/pages/VehicleList.jsx and BookingForm.jsx

```

// VehicleList.jsx
import { useEffect, useState } from 'react';
import api from '../api';

export default function VehicleList() {
  const [vehicles, setVehicles] = useState([]);
  const [form, setForm] = useState({ registration_no: '', model: '' });

  const load = async () => setVehicles((await api.get('/api/vehicles')).data);
  useEffect(() => { load(); }, []);

  const add = async () => {
    await api.post('/api/vehicles', form);
    setForm({ registration_no: '', model: '' });
    load();
  };

  return (
    <div>
      <h2>My Vehicles</h2>
      <ul>{vehicles.map((v) => <li key={v.vehicle_id}>{v.registration_no} - {v.model}</li>)}</ul>
      <input placeholder="Registration no" value={form.registration_no}
        onChange={(e) => setForm({ ...form, registration_no: e.target.value })} />
      <input placeholder="Model" value={form.model}
        onChange={(e) => setForm({ ...form, model: e.target.value })} />
      <button onClick={add}>Add Vehicle</button>
    </div>
  );
}

// BookingForm.jsx
export function BookingForm() {
  const [vehicles, setVehicles] = useState([]);
  const [services, setServices] = useState([]);
  const [form, setForm] = useState({ vehicle_id: '', service_id: '', booking_date: '' });

  useEffect(() => {
    api.get('/api/vehicles').then((r) => setVehicles(r.data));
    api.get('/api/services').then((r) => setServices(r.data));
  }, []);

  const book = async () => {

```

```

    await api.post('/api/bookings', form);
    alert('Service booked');
  };
  /* select boxes for vehicle and service, a date input and a Book button */
}

```

Listing 6.19: src/pages/StatusTracker.jsx

```

import { useEffect, useState } from 'react';
import api from '../api';

const label = { booked: 'Booked', in_progress: 'In progress', completed: 'Completed' };

export default function StatusTracker() {
  const [bookings, setBookings] = useState([]);

  useEffect(() => {
    api.get('/api/bookings').then((r) => setBookings(r.data));
  }, []);

  return (
    <table>
      <thead>
        <tr><th>Vehicle</th><th>Service</th><th>Date</th><th>Status</th></tr>
      </thead>
      <tbody>
        {bookings.map((b) => (
          <tr key={b.booking_id}>
            <td>{b.registration_no}</td>
            <td>{b.service_name}</td>
            <td>{new Date(b.booking_date).toLocaleDateString()}</td>
            <td>{label[b.status]}</td>
          </tr>
        ))}
      </tbody>
    </table>
  );
}

```

6.3 API Integration

The React front end calls the Express REST APIs using asynchronous HTTP requests and renders the JSON responses. The backend integrates two external services, Razorpay and Firebase Cloud Messaging. The complete list of APIs follows. Endpoints marked 'Customer' require a customer token and endpoints marked 'Service center' require a staff token.

API 1: POST /api/auth/register

Access	Public
Description	Register a new user. The password is hashed with bcrypt.
Request	{ "name": "A. Rao", "email": "rao@example.com", "phone": "90000000010", "password": "Secret@123" }
Success response	201 Created: { "message": "Registered", "customer_id": 1 }
Errors	400 validation error; 409 email already exists

API 2: POST /api/auth/login

Access	Public
Description	Verify credentials and issue a JWT.
Request	{ "email": "rao@example.com", "password": "Secret@123" }
Success response	200 OK: { "token": "<jwt>", "user": { "customer_id": 1, "name": "A. Rao", "role": "customer" } }
Errors	401 invalid credentials; 429 too many requests

API 3: POST /api/auth/device-token

Access	Customer
Description	Save the FCM device token used for notifications.
Request	{ "fcm_token": "<device-token>" }
Success response	201 Created: { "message": "Token saved" }
Errors	401 unauthenticated

API 4: GET /api/vehicles

Access	Customer
Description	List the vehicles of the logged-in customer.
Request	(none)
Success response	200 OK: [{ "vehicle_id": 1, "registration_no": "TS09AB1234", "model": "Swift" }]
Errors	401 unauthenticated

API 5: POST /api/vehicles

Access	Customer
Description	Add a vehicle for the logged-in customer.
Request	{ "registration_no": "TS09AB1234", "model": "Swift" }
Success response	201 Created: { "vehicle_id": 1 }
Errors	400 validation error; 409 duplicate registration number

API 6: GET /api/vehicles/:id/history

Access	Customer, Service center
Description	Service history of a vehicle (join of booking, service, job card and invoice).
Request	(none)
Success response	200 OK: [{ "booking_date": "2025-10-01", "service_name": "Oil Change", "status": "completed", "total_amount": 1500.00 }]
Errors	403 not the owner; 404 vehicle not found

API 7: GET /api/services

Access	Any logged-in user
Description	List the services offered with base prices.

Access	Any logged-in user
Request	(none)
Success response	200 OK: [{ "service_id": 2, "service_name": "Oil Change", "base_price": 800.00 }]
Errors	401 unauthenticated

API 8: POST /api/bookings

Access	Customer
Description	Create a service booking for one of the customer's vehicles.
Request	{ "vehicle_id": 1, "service_id": 2, "booking_date": "2025-10-10" }
Success response	201 Created: { "booking_id": 7, "status": "booked" }
Errors	400 validation error; 403 vehicle not owned; 404 service not found

API 9: GET /api/bookings

Access	Customer, Service center
Description	List bookings. Customers see their own; the service center can filter by date and status.
Request	Query: ?date=2025-10-10&status=booked
Success response	200 OK: [{ "booking_id": 7, "registration_no": "TS09AB1234", "service_name": "Oil Change", "status": "booked" }]
Errors	401 unauthenticated

API 10: PUT /api/bookings/:id/status

Access	Service center
Description	Update the status of a booking and notify the customer.
Request	{ "status": "completed" }
Success response	200 OK: { "booking_id": 7, "status": "completed" }
Errors	400 invalid status; 404 booking not found

API 11: GET /api/technicians

Access	Service center
Description	List technicians with availability.
Request	(none)
Success response	200 OK: [{ "technician_id": 1, "name": "Technician One", "is_available": 1 }]
Errors	403 access denied

API 12: POST /api/jobcards

Access	Service center
Description	Create a job card for a booking and allocate a technician (transaction).
Request	{ "booking_id": 7, "technician_id": 1 }
Success response	201 Created: { "job_card_id": 4 }

Access	Service center
Errors	404 booking not found; 409 job card already exists

API 13: POST /api/jobcards/:id/parts

Access	Service center
Description	Record a spare part used on a job card and reduce the stock (transaction).
Request	{ "part_id": 1, "quantity": 2 }
Success response	201 Created: { "message": "Part added" }
Errors	400 insufficient stock; 404 part not found

API 14: POST /api/invoices

Access	Service center
Description	Generate the invoice for a job card: service base price plus parts used.
Request	{ "job_card_id": 4 }
Success response	201 Created: { "invoice_id": 3, "total_amount": 1700.00, "status": "unpaid" }
Errors	409 invoice already exists

API 15: GET /api/invoices/:id

Access	Customer, Service center
Description	View an invoice with its line items.
Request	(none)
Success response	200 OK: { "invoice_id": 3, "total_amount": 1700.00, "status": "unpaid", "items": [...] }
Errors	403 not the owner; 404 not found

API 16: POST /api/payments

Access	Customer
Description	Create a Razorpay order for an unpaid invoice.
Request	{ "invoice_id": 3 }
Success response	201 Created: { "order_id": "order_xxx", "amount": 170000, "key_id": "rzp_test_xxx" }
Errors	409 invoice already paid

API 17: POST /api/payments/verify

Access	Customer
Description	Verify the Razorpay signature and mark the payment and invoice as paid.
Request	{ "razorpay_order_id": "order_xxx", "razorpay_payment_id": "pay_xxx", "razorpay_signature": "<sig>" }
Success response	200 OK: { "message": "Payment successful" }
Errors	400 signature mismatch

Razorpay integration

The backend creates a payment order for an invoice; after the customer pays, the signature is verified, the payment details are stored in the Payment table and the invoice is marked paid. The sequence is shown below.

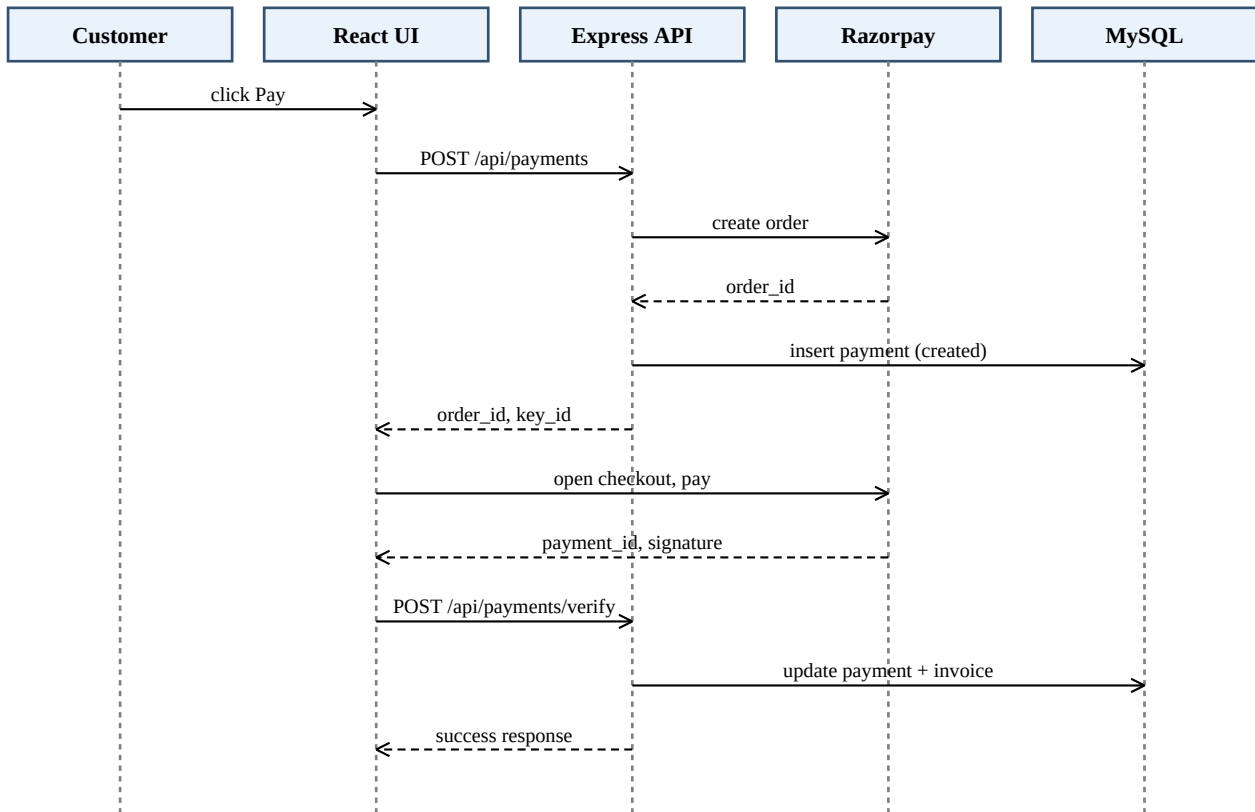


Figure: Online payment sequence

Listing 6.20: `src/services/razorpay.service.js` and `payment.service.js`

```

const Razorpay = require('razorpay');
const crypto = require('crypto');
const db = require('../config/db');

const razorpay = new Razorpay({
  key_id: process.env.RAZORPAY_KEY_ID,
  key_secret: process.env.RAZORPAY_KEY_SECRET,
});

exports.createOrder = async (user, invoiceId) => {
  const [[inv]] = await db.query(
    `SELECT i.total_amount, i.status FROM Invoice i
     JOIN Job_Card j ON j.job_card_id = i.job_card_id
     JOIN Booking b ON b.booking_id = j.booking_id
     JOIN Vehicle v ON v.vehicle_id = b.vehicle_id
     WHERE i.invoice_id = ? AND v.customer_id = ?`, [invoiceId, user.id]);
  if (!inv) throw Object.assign(new Error('Invoice not found'), { status: 404 });
  if (inv.status === 'paid') throw Object.assign(new Error('Already paid'), { status: 409 });

  const order = await razorpay.orders.create({
    amount: Math.round(inv.total_amount * 100), // paise
    currency: 'INR',
    receipt: `inv_${invoiceId}`,
  });

  await db.query(
    `INSERT INTO Payment (invoice_id, amount, razorpay_order_id) VALUES (?, ?, ?)`,
  );
}

```

```

    [invoiceId, inv.total_amount, order.id]
  );
  return { order_id: order.id, amount: order.amount, key_id: process.env.RAZORPAY_KEY_ID };
};

exports.verify = async ({ razorpay_order_id, razorpay_payment_id, razorpay_signature }) => {
  const expected = crypto
    .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
    .update(`${razorpay_order_id}|${razorpay_payment_id}`)
    .digest('hex');
  if (expected !== razorpay_signature) {
    await db.query("UPDATE Payment SET payment_status='failed' WHERE razorpay_order_id=?",
      [razorpay_order_id]);
    throw Object.assign(new Error('Signature mismatch'), { status: 400 });
  }
  const conn = await db.getConnection();
  try {
    await conn.beginTransaction();
    await conn.query(
      `UPDATE Payment SET payment_status='success', razorpay_payment_id=?, paid_at=NOW()
      WHERE razorpay_order_id=?`, [razorpay_payment_id, razorpay_order_id]);
    await conn.query(
      `UPDATE Invoice SET status='paid' WHERE invoice_id =
      (SELECT invoice_id FROM Payment WHERE razorpay_order_id=?`, [razorpay_order_id]);
    await conn.commit();
  } catch (e) { await conn.rollback(); throw e; } finally { conn.release(); }
};

```

Listing 6.21: src/pages/InvoiceView.jsx (Razorpay checkout in the browser)

```

import { useEffect, useState } from 'react';
import { useParams } from 'react-router-dom';
import api from '../api';

export default function InvoiceView() {
  const { id } = useParams();
  const [invoice, setInvoice] = useState(null);

  useEffect(() => { api.get(`/api/invoices/${id}`).then((r) => setInvoice(r.data)); }, [id]);

  const pay = async () => {
    const { data } = await api.post('/api/payments', { invoice_id: id });
    const rzp = new window.Razorpay({
      key: data.key_id,
      amount: data.amount,
      currency: 'INR',
      order_id: data.order_id,
      name: 'Vehicle Service Center',
      handler: async (response) => {
        await api.post('/api/payments/verify', response);
        alert('Payment successful');
        window.location.reload();
      },
    });
    rzp.open();
  };

  if (!invoice) return <p>Loading...</p>;
  return (
    <div>
      <h2>Invoice #{invoice.invoice_id}</h2>
      <p>Total: Rs. {invoice.total_amount} ({invoice.status})</p>
      {invoice.status === 'unpaid' && <button onClick={pay}>Pay Online</button>}
    </div>
  );
}

```

Firestore Cloud Messaging integration

The backend sends a notification to the customer when the booking status changes and when an invoice is ready. If a notification cannot be delivered, the error is logged and the main operation is not affected.

Listing 6.22: src/services/fcm.service.js

```
const admin = require('firebase-admin');
const db = require('../config/db');

admin.initializeApp({
  credential: admin.credential.cert(require(process.env.FIREBASE_KEY_FILE)),
});

exports.notifyCustomer = async (customerId, title, body) => {
  try {
    const [rows] = await db.query(
      'SELECT fcm_token FROM Device_Token WHERE customer_id = ?', [customerId]);
    if (!rows.length) return;
    await admin.messaging().sendEachForMulticast({
      tokens: rows.map((r) => r.fcm_token),
      notification: { title, body },
    });
  } catch (err) {
    console.error('FCM error:', err.message); // notification failure must not break the flow
  }
};
```

6.4 Authentication and Authorisation

A user registers with name, email, phone and password. The password is hashed with bcrypt and only the hash is stored (Listing 6.9). At login the backend compares the password with the stored hash and, if it matches, issues a signed JWT containing the user id and role. Protected routes use the verifyToken middleware (Listing 6.10) that verifies the token. Role-based access control then decides what each user may do through the authorize middleware: a customer can manage only their own vehicles, bookings and invoices, while service center staff can manage all bookings, job cards, technicians, parts, invoices and payments. Requests without a valid token, or with the wrong role, are rejected with 401 or 403.

Threat	Protection used
Stolen database	Passwords stored only as bcrypt hashes
Forged requests	Tokens signed with JWT_SECRET and verified on every request
Access to other users' data	Role checks and ownership checks (customer_id comparison) in the service layer
SQL injection	Parameterised queries (? placeholders) throughout
Brute-force login	Strict rate limit on the login route
Malformed input	express-validator rules on every write endpoint
Eavesdropping	HTTPS for all traffic; secure headers through helmet
Leaked secrets	Keys in environment variables, never in source control

CHAPTER 7- FEATURES

7.1 List of Main Features

No.	Feature	Main users
1	Customer registration and secure login (JWT)	Customer, Service center
2	Vehicle registration and management	Customer
3	Online service booking	Customer
4	Job-card creation and technician allocation	Service center
5	Service-status tracking (booked, in progress, completed)	Customer, Service center
6	Spare-part management linked to job cards	Service center
7	Digital invoice generation	Service center
8	Online payment through Razorpay	Customer
9	Notifications through Firebase Cloud Messaging	Customer
10	Complete service history of every vehicle	Customer, Service center
11	Role-based access for customers and the service center	All users

7.2 How Each Feature Works

Each feature is described from the user's point of view and then technically, with the APIs and tables involved.

1. Registration and Secure Login

Used by: Customer. User perspective: Users create an account and log in with email and password. After login the user stays signed in until the token expires.

Technical description: POST /api/auth/register stores the user with a bcrypt-hashed password. POST /api/auth/login compares the password with the hash and returns a JWT (8 hours) containing the user id and role. A strict rate limit protects the login route.

2. Vehicle Registration and Management

Used by: Customer. User perspective: Customers add their vehicles with registration number and model, and can see the list of their vehicles at any time.

Technical description: GET and POST /api/vehicles read and write the Vehicle table using the customer id from the token. The UNIQUE constraint on registration_no prevents the same vehicle from being registered twice.

3. Online Service Booking

Used by: Customer. User perspective: The customer chooses one of their vehicles, a service from the list and a preferred date, and submits the booking.

Technical description: POST /api/bookings validates the input, checks that the vehicle belongs to the customer and that the service exists, and inserts a Booking row with status 'booked'. The indexes on booking_date and status keep later listing queries fast.

4. Job-Card Creation and Technician Allocation

Used by: Service center. User perspective: The service center opens a booking, creates a job card and assigns an available technician.

Technical description: POST /api/jobcards runs a transaction: it locks the booking row, inserts a Job_Card (UNIQUE on booking_id guarantees one job card per booking) and sets the booking status to 'in_progress'. If any step fails the transaction is rolled back.

5. Service-Status Tracking

Used by: Customer, Service center. User perspective: The customer can see whether the vehicle is booked, in progress or completed without calling the center.

Technical description: PUT /api/bookings/:id/status updates the status (allowed values are checked by validation). The customer's status page reads the same Booking row through GET /api/bookings, so both sides always see the same information.

6. Spare-Part Management

Used by: Service center. User perspective: The service center records the parts used on a vehicle and the stock is updated automatically.

Technical description: POST /api/jobcards/:id/parts locks the Spare_Part row, checks that enough stock exists, inserts a Job_Card_Part row with the price at the time of use and reduces stock_qty, all in one transaction.

7. Digital Invoice Generation

Used by: Service center. User perspective: When the work is finished the system produces an invoice that includes the service price and all parts used.

Technical description: POST /api/invoices adds the base price of the service and the sum of quantity x unit_price from Job_Card_Part, stores an Invoice with status 'unpaid', marks the booking completed and closes the job card, in one transaction.

8. Online Payment

Used by: Customer. User perspective: The customer pays the invoice online using the Razorpay checkout window.

Technical description: POST /api/payments creates a Razorpay order and a Payment row with status 'created'. After checkout, POST /api/payments/verify checks the HMAC signature with the secret key, marks the payment 'success' and the invoice 'paid' in one transaction.

9. Notifications

Used by: Customer. User perspective: The customer is notified when the status of a booking changes and when an invoice is ready.

Technical description: After the relevant database update, the backend reads the customer's tokens from Device_Token and sends a message through Firebase Cloud Messaging. Delivery errors are logged and do not stop the main operation.

10. Service History

Used by: Customer, Service center. User perspective: Customers and staff can open any vehicle and see every past service with its status and amount.

Technical description: GET /api/vehicles/:id/history runs a join across Booking, Service, Job_Card and Invoice, ordered by date. Customers can open only their own vehicles.

11. Security and Role-Based Access

Used by: All users. User perspective: Each user sees and does only what their role allows.

Technical description: The verifyToken and authorize middleware, ownership checks, validation, rate limiting, helmet headers and HTTPS protect the system. SQL is always parameterised.

CHAPTER -8 TESTING

8.1 Postman Testing (Frontend/Backend)

Postman is used to test the REST APIs independently of the React interface. For every endpoint a request is sent with valid and invalid data and the response is checked for the correct status code, response body and database change. Tests cover authentication (register, login, missing or invalid token), validation (missing fields, duplicate email or registration number), role-based access (a customer calling a service-center-only endpoint) and the booking, invoice and payment endpoints.

The requests are organised into a Postman collection with one folder per module. A collection variable holds the base URL and another holds the JWT, which is saved automatically after a successful login so that later requests can use it.

Folder	Requests
Auth	Register, Login (customer), Login (service center), Save device token
Vehicles	Add vehicle, List vehicles, Vehicle history
Services	List services
Bookings	Create booking, List bookings, Update status
Job cards	Create job card, Add part
Invoices	Generate invoice, View invoice
Payments	Create order, Verify payment

Listing 8.1: Postman test script for the login request

```
pm.test("Status code is 200", function () {
  pm.response.to.have.status(200);
});

pm.test("Response contains a token and role", function () {
  const body = pm.response.json();
  pm.expect(body).to.have.property("token");
  pm.expect(body.user).to.have.property("role");
  pm.collectionVariables.set("token", body.token); // reused by later requests
});

pm.test("Response time is acceptable", function () {
  pm.expect(pm.response.responseTime).to.be.below(1000);
});
```

The front end is tested by using the React screens against the running backend: logging in, adding a vehicle, booking a service and checking that the status shown to the customer matches the database.

8.2 Integration Testing

Integration testing checks that the modules work together. The end-to-end flow tested is: register, log in, add a vehicle, book a service, create a job card and allocate a technician, update the status, generate an invoice and record a payment. At every step the related tables are checked in MySQL to confirm that foreign keys are correct and that no orphan or duplicate records exist.

Step	Action	Database check
1	Register and log in as a customer	One row in Customer with a password hash
2	Add a vehicle	Vehicle row with the customer's id
3	Book a service	Booking row with status 'booked'
4	Log in as service center; create job card	Job_Card row; Booking status 'in_progress'
5	Add spare parts	Job_Card_Part rows; Spare_Part stock reduced
6	Generate invoice	Invoice row; Booking status 'completed'; Job_Card.closed_at set
7	Pay through Razorpay	Payment row 'success'; Invoice status 'paid'
8	Open vehicle history	History query returns the completed service

The following queries are used to verify data integrity after the tests.

Listing 8.2: integrity checks

```
-- Bookings without a vehicle (should return 0 rows)
SELECT b.booking_id FROM Booking b
LEFT JOIN Vehicle v ON v.vehicle_id = b.vehicle_id WHERE v.vehicle_id IS NULL;

-- Invoices without a job card (should return 0 rows)
SELECT i.invoice_id FROM Invoice i
LEFT JOIN Job_Card j ON j.job_card_id = i.job_card_id WHERE j.job_card_id IS NULL;

-- Duplicate job cards for one booking (should return 0 rows)
SELECT booking_id, COUNT(*) FROM Job_Card GROUP BY booking_id HAVING COUNT(*) > 1;

-- Paid invoices without a successful payment (should return 0 rows)
SELECT i.invoice_id FROM Invoice i
WHERE i.status = 'paid'
AND NOT EXISTS (SELECT 1 FROM Payment p
                WHERE p.invoice_id = i.invoice_id AND p.payment_status = 'success');
```

8.3 Tools Used for Testing

- Postman: API request testing, saved collections, automatic checks on status codes and response bodies.
- MySQL Workbench: checking table contents, constraints, query results and index usage with EXPLAIN.
- Browser developer tools: checking front-end requests, responses and console errors.
- Git and GitHub: tracking fixes made after failed tests.

8.4 Test Cases and Results

The database was tested with sample customers, vehicles, services and bookings. The test cases below cover authentication, validation, security, the booking and service workflow, invoicing, payments, notifications and data integrity. The Result column is to be filled in with Pass or Fail when each case is executed.

ID	Module	Test case	Input / Action	Expected result	Result
TC01	Registration	Register with valid details	POST /api/auth/register with name, email, phone, password	201 Created; password stored as hash	[]
TC02	Registration	Register with an existing email	Same email as an existing user	409 Conflict	[]
TC03	Registration	Register with a short password	Password of 3 characters	400 validation error	[]
TC04	Registration	Register with invalid email	email = 'abc'	400 validation error	[]
TC05	Login	Login with correct credentials	Valid email and password	200 OK with JWT and user role	[]
TC06	Login	Login with wrong password	Valid email, wrong password	401 Unauthorized	[]
TC07	Login	Login with unknown email	Email not registered	401 Unauthorized	[]
TC08	Login	Repeated failed logins	More than 10 attempts in 15 minutes	429 Too Many Requests	[]
TC09	Security	Access protected route without token	GET /api/vehicles with no Authorization header	401 Unauthorized	[]
TC10	Security	Access with an invalid token	Modified token string	401 Unauthorized	[]
TC11	Security	Customer calls a staff-only route	PUT /api/bookings/1/status with customer token	403 Forbidden	[]
TC12	Security	Customer reads another customer's vehicle history	GET /api/vehicles/:id/history of another user	403 Forbidden	[]
TC13	Security	SQL injection attempt in a field	registration_no = "x"; DROP TABLE Vehicle;--"	Treated as plain text; no table is affected	[]
TC14	Vehicle	Add a vehicle	Valid registration_no and model	201 Created with vehicle_id	[]
TC15	Vehicle	Add a vehicle with duplicate registration number	Existing registration_no	409 Conflict (UNIQUE constraint)	[]
TC16	Vehicle	Add a vehicle with missing model	model omitted	400 validation error	[]
TC17	Vehicle	List own vehicles	GET /api/vehicles	Only the customer's vehicles are returned	[]
TC18	Booking	Create a booking	Valid vehicle, service, date	201 Created; status = 'booked'	[]
TC19	Booking	Book with someone else's vehicle	vehicle_id owned by another customer	403 Forbidden	[]

ID	Module	Test case	Input / Action	Expected result	Result
TC20	Booking	Book with an unknown service	service_id = 9999	404 Not Found	[]
TC21	Booking	Book with an invalid date	booking_date = 'tomorrow'	400 validation error	[]
TC22	Booking	List bookings by date and status (staff)	?date=...&status=booked	Only matching bookings returned	[]
TC23	Status	Update status to 'in_progress'	PUT /api/bookings/:id/status	200 OK; customer sees the new status	[]
TC24	Status	Update status to an invalid value	status = 'finished'	400 validation error	[]
TC25	Status	Update status of unknown booking	id = 9999	404 Not Found	[]
TC26	Job card	Create a job card with a technician	booking_id, technician_id	201 Created; booking becomes 'in_progress'	[]
TC27	Job card	Create a second job card for the same booking	Same booking_id	409 Conflict (UNIQUE booking_id)	[]
TC28	Job card	Rollback on failure	Invalid technician_id	No job card created and booking status unchanged	[]
TC29	Spare parts	Add a part with enough stock	part_id, quantity within stock	201 Created; stock reduced by quantity	[]
TC30	Spare parts	Add a part with insufficient stock	quantity greater than stock	400 Insufficient stock; stock unchanged	[]
TC31	Invoice	Generate an invoice	job_card_id of a job with parts	Total = base price + sum(quantity x unit_price)	[]
TC32	Invoice	Generate a duplicate invoice	Same job_card_id again	409 Conflict (UNIQUE job_card_id)	[]
TC33	Invoice	View own invoice	GET /api/invoices/:id	200 OK with line items	[]
TC34	Payment	Create a payment order	POST /api/payments for unpaid invoice	201 Created with order_id and key_id	[]
TC35	Payment	Create an order for a paid invoice	Invoice already paid	409 Conflict	[]
TC36	Payment	Verify with a correct signature	Valid Razorpay response values	200 OK; payment 'success'; invoice 'paid'	[]
TC37	Payment	Verify with a wrong signature	Altered signature	400 error; payment 'failed'	[]
TC38	Notification	Notification on status change	Update a booking status	FCM message sent to customer's device	[]
TC39	Notification	Notification failure handling	Invalid FCM token	Error logged; API still returns success	[]
TC40	History	View service history	GET /api/vehicles/:id/history	All services listed, newest first	[]
TC41	Integrity	Delete a customer who owns vehicles	DELETE in MySQL	Rejected by foreign key constraint	[]

ID	Module	Test case	Input / Action	Expected result	Result
TC42	Integrity	Insert a negative stock quantity	stock_qty = -1	Rejected by CHECK constraint	[]
TC43	End-to-end	Complete flow	Register, book, job card, status, invoice, pay	All records created with correct links; history shows the service	[]

Performance evaluation

Parameter	Approach	Observed Result
API response time	Postman timing on key endpoints	[fill in] ms
Query performance	EXPLAIN and timing of the booking list query with indexes on booking_date and status	[fill in] ms before / [fill in] ms after
Security	JWT, RBAC, hashing, rate limiting	Unauthorised requests rejected
Data integrity	PK/FK constraints, transactions, integrity queries	No orphan or duplicate records
Load test	Concurrent booking requests	[fill in] requests/sec

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

- 1 Install Node.js and MySQL Server on the target machine.
- 2 Clone the project repository from GitHub.
- 3 Create the database and run schema.sql to create all tables, constraints and indexes.
- 4 Run seed.sql to insert the initial data (services, technicians, spare parts) and create a service-center user.
- 5 Create the .env file with the database and secret settings (section 9.2).
- 6 In the backend folder run npm install and start the Express server with npm start.
- 7 In the frontend folder run npm install, then npm start (development) or npm run build (production).
- 8 Open the application in a browser and test login and booking.

Listing 9.1: commands

```
# 1. Get the code
git clone <repository-url>
cd vehicle-service

# 2. Create the database
mysql -u root -p < database/schema.sql
mysql -u root -p vehicle_service < database/seed.sql

# 3. Start the backend
cd backend
npm install
npm start                # runs on http://localhost:5000

# 4. Start the frontend (new terminal)
cd frontend
npm install
npm start                # runs on http://localhost:3000
```

9.2 Environment Configuration

Sensitive settings are kept in an environment file and are never committed to GitHub. The values needed are:

Variable	Purpose
DB_HOST, DB_USER, DB_PASSWORD, DB_NAME	MySQL connection details
JWT_SECRET	Secret used to sign and verify JWTs
RAZORPAY_KEY_ID, RAZORPAY_KEY_SECRET	Razorpay payment gateway credentials
FIREBASE_KEY_FILE	Path to the Firebase service account file for FCM

Variable	Purpose
CLIENT_URL	Allowed front-end origin for CORS
PORT	Port on which the Express server runs

Listing 9.2: backend/.env.example

```
PORT=5000
CLIENT_URL=http://localhost:3000

DB_HOST=localhost
DB_USER=root
DB_PASSWORD=change_me
DB_NAME=vehicle_service

JWT_SECRET=change_this_to_a_long_random_string

RAZORPAY_KEY_ID=rzp_test_xxxxxxx
RAZORPAY_KEY_SECRET=xxxxxxxxxxxxxxxxx

FIREBASE_KEY_FILE=./firebase-service-account.json
```

9.3 Hosting of Frontend and Backend

At present the application runs on a local machine. The built React files can be served as static files, and the Express backend runs as a Node.js service connected to MySQL. Docker deployment, which packages the backend and database into containers, is the next planned step. A planned configuration is shown below. For public hosting, the front end, backend and database can be placed on any cloud provider, with HTTPS enabled and the environment variables configured on the server.

Listing 9.3: planned Dockerfile and docker-compose.yml

```
# backend/Dockerfile
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
EXPOSE 5000
CMD ["npm", "start"]

# docker-compose.yml
services:
  db:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_PASSWORD}
      MYSQL_DATABASE: vehicle_service
    volumes:
      - dbdata:/var/lib/mysql
      - ./database/schema.sql:/docker-entrypoint-initdb.d/schema.sql
  backend:
    build: ./backend
    env_file: ./backend/.env
    environment:
      DB_HOST: db
    ports:
      - "5000:5000"
    depends_on:
      - db
volumes:
  dbdata:
```

Deployment checklist

- Use HTTPS and set a strong JWT_SECRET.
- Use Razorpay live keys only on the production server and keep test keys for development.
- Create a separate MySQL user with only the privileges the application needs.
- Take regular backups of the database, for example with mysqldump.
- Do not commit .env or the Firebase key file to GitHub.
- Keep the dependencies updated and review the server logs regularly.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

Designing a consistent schema

The first version of the design stored customer, vehicle and service details together in a single booking record. This caused repeated data and made updates risky: changing a customer's phone number meant changing many rows. Deciding which attributes belong to which entity, and how entities relate (one-to-many, one-to-one and many-to-many), took several revisions of the ER model.

Slow queries on bookings

The most common screen of the service center lists bookings by date and status. As the number of test bookings grew, this query had to scan the whole Booking table, and the response became noticeably slower.

Securing user access

Customers must never see each other's vehicles, bookings or invoices, and only the service center may change statuses or create invoices. Checking this in every controller would have been repetitive and easy to forget.

Handling payments and notifications

Razorpay and FCM are external services that can fail or respond slowly. A payment can be started but never completed, and a notification can fail because a device token is no longer valid. These failures must not leave the database in an inconsistent state.

Keeping the backend maintainable

As endpoints were added, route files began to contain validation, SQL and response code together, which made them long and hard to read and change.

10.2 Solutions Applied

Challenge	Solution
Designing a consistent schema	Normalized to 3NF with PK/FK constraints; separate tables for each entity and for the many-to-many link Job_Card_Part
Slow queries on bookings	Added indexes on booking date and status and verified the improvement with EXPLAIN
Securing user access	JWT, RBAC, password hashing, validation; central verifyToken and authorize middleware plus ownership checks in the service layer
Handling payments and notifications	Integrated Razorpay and FCM with error handling; signature verification; payments stored with a status; related updates in transactions
Keeping backend maintainable	Layered router-controller-service structure and a single error handler

Each solution was tested after it was applied. For example, after the indexes were added the booking-list query was run again with EXPLAIN to confirm that the indexes were used, and after the middleware

was introduced the role tests in Chapter 8 were repeated.

10.3 Current Limitations or Known Bugs

- The job-card and technician modules, invoice generation, payment integration, notifications and front-end integration are still being completed.
- Testing and load testing are pending, so performance figures are not final.
- Docker deployment is not yet done.
- The system is designed for one service center and does not support multiple branches.
- Notifications and online payment need an internet connection.
- Real-time status updates are not available; the customer sees the latest status when the page is refreshed or a notification arrives.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

Real-time vehicle service tracking

At present the customer sees the status when the page is loaded or a notification arrives. Real-time tracking would push every change to the customer's screen immediately, and could show finer steps such as 'washing' or 'quality check'.

Vector search for smart service recommendations

Service histories and descriptions can be converted into vectors so that the system can recommend services similar to those that other vehicles of the same model needed at a similar mileage.

Microservices with an API gateway and database-per-service

The layered backend is already separated into modules. As the system grows, Auth, Vehicles, Bookings, Invoices and Payments can become separate services behind an API gateway, each with its own database, so they can be scaled and deployed independently.

11.2 Possible Integrations or Optimisations

Redis caching

Frequently read and rarely changed data, such as the list of services and prices, can be cached in Redis to give faster responses and reduce load on MySQL.

MongoDB for logs and activity data (polyglot persistence)

Logs and activity data are written often and have a flexible structure. Storing them in MongoDB, while MySQL continues to hold the transactional data, would use each database for what it does best.

Docker, Kubernetes and CI/CD

Containers make deployment repeatable. Kubernetes can run several copies of the backend, and a CI/CD pipeline can run tests and deploy automatically after every change pushed to GitHub.

Monitoring with Prometheus and Grafana

Prometheus can collect metrics such as request rate, error rate and response time from the backend, and Grafana can show them on dashboards and raise alerts.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The Vehicle Service Booking and Tracking System provides a centralized, database-driven platform for managing the complete vehicle servicing process. Customers register vehicles, book services and follow progress online, while the service center manages bookings, job cards, technicians, spare parts, invoices and payments.

The system is built on a normalized MySQL database and a secure Node.js and Express.js REST backend, with a React.js front end, Razorpay payments and Firebase notifications. It offers faster and easier service booking, better service-status visibility, digital job cards and invoices, organized customer and vehicle records, improved service history management and a secure REST-based backend.

12.2 What Was Achieved

- Customers can register, log in and add vehicles.
- Service bookings are created and stored with their status.
- Job cards and technician allocation are managed by the service center.
- Service status can be updated and tracked by the customer.
- Invoices are generated and payments are recorded.
- Service history is available for every vehicle.
- A normalized 3NF schema with constraints and indexes, and a secure REST backend, were designed and built.

Feature	Existing (Manual) System	Proposed System
Booking	Phone call / visit	Online service booking
Job card	Paper job card	Digital job card
Status visibility	Limited	Real-time status tracking
Records	Paper, hard to maintain	Centralized MySQL database
Billing	Manual billing	Digital invoice and online payment
Communication	Delayed	Notifications (FCM)

12.3 Skills Learned During Development

- Relational database design: ER modelling, normalization, constraints and indexing.
- Writing SQL: DDL, joins, aggregates and transactions.

- Building secure REST APIs with Node.js and Express.js.
- Authentication and authorisation with JWT, bcrypt and RBAC.
- Integrating third-party services (Razorpay and Firebase Cloud Messaging).
- Building a React.js interface that consumes REST APIs.
- Testing with Postman, using Git and GitHub, and teamwork.

Beyond the technical skills, the project showed the importance of designing the data model carefully before writing code, because every API, screen and report depends on it. It also showed how much simpler security and maintenance become when the code is organised in layers with shared middleware.

"A database-driven platform for efficient and scalable vehicle service management."

CHAPTER -13 REFERENCES

Books

- Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems. Pearson.

Documentation and Websites

- MySQL Reference Manual: <https://dev.mysql.com/doc/>
- Node.js Documentation: <https://nodejs.org/docs/>
- Express.js Documentation: <https://expressjs.com/>
- React Documentation: <https://react.dev/>
- Razorpay API Documentation: <https://razorpay.com/docs/>
- Firebase Cloud Messaging: <https://firebase.google.com/docs/cloud-messaging>
- JWT Introduction: <https://jwt.io/introduction>

Research

- Research papers and articles on vehicle service management systems.

CHAPTER -14 APPENDICES

Screenshots of the app

The following pages are reserved for screenshots of the working application.

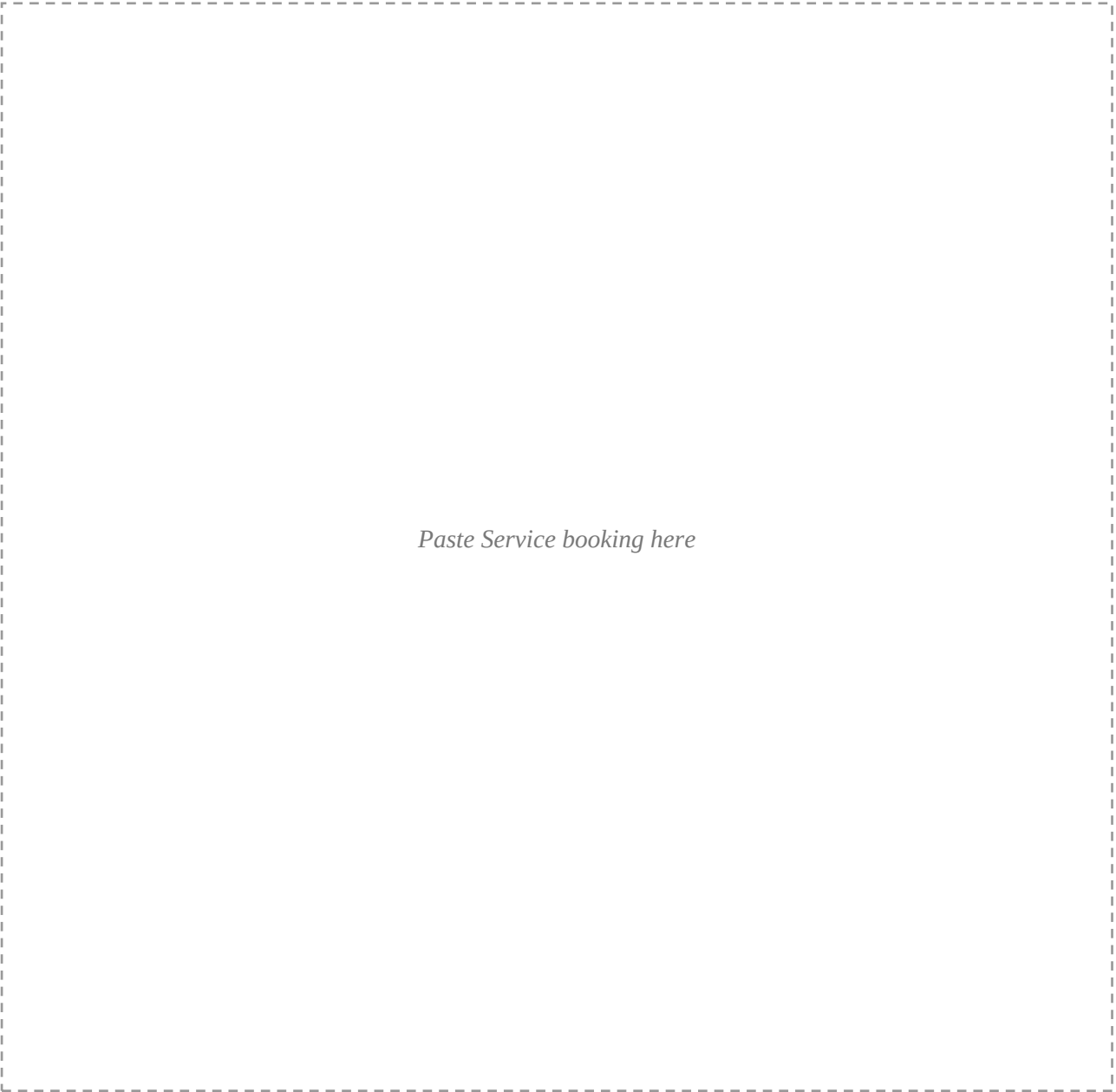
Screenshot 1: Login page

Paste Login page here

Screenshot 2: Vehicle registration

Paste Vehicle registration here

Screenshot 3: Service booking



Screenshot 4: Job card / status tracking

Paste Job card / status tracking here

Screenshot 5: Invoice and payment

Paste Invoice and payment here

Sample code snippets

The main code listings of the project are given in Chapter 5 (SQL) and Chapter 6 (backend and front end). A further sample, the Postman test script, is in Chapter 8.

Installation/setup instructions

The Vehicle Service Booking and Tracking System needs Node.js and MySQL. Follow these steps.

- 1 Install Node.js (LTS version) and MySQL Server with MySQL Workbench.
- 2 Download or clone the project from GitHub.
- 3 Open MySQL Workbench and run database/schema.sql, then database/seed.sql.
- 4 In the backend folder copy .env.example to .env and fill in the database password, JWT secret, Razorpay keys and the Firebase key file path.
- 5 Run npm install and npm start in the backend folder.
- 6 Run npm install and npm start in the frontend folder.
- 7 Open <http://localhost:3000> in a browser.
- 8 Use Postman to test the APIs if required, with the collection described in Chapter 8.

User manual or guide

For customers

- 1 Open the application and choose Register. Enter your name, email, phone number and a password of at least 8 characters.
- 2 Log in with your email and password.
- 3 Open My Vehicles and add each vehicle with its registration number and model.
- 4 Open Book Service, choose the vehicle, the service and the date, and press Book.
- 5 Open Status to see whether your vehicle is booked, in progress or completed. You will also receive a notification when the status changes.
- 6 When the invoice is ready, open it and press Pay Online. Complete the payment in the Razorpay window.
- 7 Open the vehicle's history at any time to see all past services and amounts.

For the service center

- 1 Log in with a service center account.
- 2 Open Bookings and filter by date and status to see the work for the day.
- 3 Open a booking, press Create Job Card and choose an available technician. The status changes to In progress.
- 4 While the work is done, add the spare parts used to the job card. The stock is reduced automatically.
- 5 When the work is complete, press Generate Invoice. The booking is marked Completed.
- 6 Check the payment status of each invoice. Use the reports to see bookings per service and revenue per day.

Troubleshooting

Problem	What to check
Cannot log in	Check the email and password; wait 15 minutes if you made many failed attempts
Server does not start	Check that MySQL is running and the .env values are correct
Payment window does not open	Check the internet connection and the Razorpay key in .env
No notification received	Check that notifications are allowed in the browser and the device token was saved

Geo Tag photos with guide

Geo-tag photo 1: Team with guide

Paste Team with guide here

Geo-tag photo 2: Team at work

Paste Team at work here

Review forms with guide signatures

Review form 1: Review 1 form with guide signature

Paste Review 1 form with guide signature here

Review form 2: Review 2 form with guide signature

Paste Review 2 form with guide signature here